

Data Preprocessing (ডেটা প্রিপ্রসেসিং)

ভূমিকা (Introduction)

ডেটা মাইনিং (Data Mining) বা মেশিন লার্নিং (Machine Learning)-এ কাজ করার আগে সবচেয়ে গুরুত্বপূর্ণ ধাপগুলোর একটি হলো **Data Preprocessing** (ডেটা প্রিপ্রসেসিং)। বাস্তব জীবনের (real-world) ডেটা সাধারণত সরাসরি ব্যবহারযোগ্য থাকে না। এই ডেটা অনেক সময় অসম্পূর্ণ, ভুল বা অসামঞ্জস্যপূর্ণ হয়। তাই বিশ্লেষণের আগে ডেটাকে পরিষ্কার, সাজানো এবং সঠিক করা প্রয়োজন।

👉 সহজভাবে বলতে গেলে,

Data Preprocessing = Raw data → Clean & usable data

Data Quality: Why Preprocess the Data? (কেন ডেটা প্রিপ্রসেসিং প্রয়োজন?)

বাস্তব জীবনের ডেটা সাধারণত “dirty data” হয়, অর্থাৎ এতে অনেক সমস্যা থাকে। এই সমস্যাগুলো তিন ধরনের হতে পারে:

1. Incomplete Data (অসম্পূর্ণ ডেটা)

যখন ডেটার কিছু অংশ বা attribute missing থাকে, তখন তাকে incomplete data বলে।

📌 উদাহরণ:

- occupation = “ ” (খালি)
- customer age নেই
- height attribute নেই

👉 অর্থাৎ, প্রয়োজনীয় তথ্যই পাওয়া যাচ্ছে না।

2. Noisy Data (ত্রুটিযুক্ত বা ভুল ডেটা)

যখন ডেটায় ভুল মান (error) বা অস্বাভাবিক মান (outlier) থাকে।

📌 উদাহরণ:

- Salary = “-10” (অসম্ভব মান)
- temperature = 999°C (ভুল মাপ)

👉 এগুলো বাস্তবসম্মত নয়, তাই model ভুল সিদ্ধান্ত নিতে পারে।

3. Inconsistent Data (অসামঞ্জস্যপূর্ণ ডেটা)

যখন একই তথ্য ভিন্নভাবে সংরক্ষিত থাকে বা একে অপরের সাথে মিল (Discrepancy between duplicate records) না থাকে।

📌 উদাহরণ:

- Age = 42, কিন্তু Birthday = 03/07/1997 ❌ (মিলছে না)
- Rating আগে ছিল: 1,2,3 → এখন A,B,C
- duplicate record-এ ভিন্ন ভিন্ন তথ্য

👉 এতে data conflict তৈরি হয়।

? Why Incomplete? (ডেটা অসম্পূর্ণ কেন হয়?)

ডেটা অসম্পূর্ণ হওয়ার প্রধান কারণগুলো হলো:

📌 1. Data Collection Problem

সব attribute সবসময় সংগ্রহ করা হয় না

👉 উদাহরণ: customer-এর age নেওয়া হয়নি

📌 2. Unimportant মনে করা

কিছু তথ্য তখন গুরুত্বপূর্ণ মনে হয়নি

👉 উদাহরণ: height নেওয়া হয়নি

📌 3. Device Malfunction

ডেটা সংগ্রহের যন্ত্র ঠিকমতো কাজ করেনি (Entry te vul)

📌 4. Data Removal

অসামঞ্জস্যপূর্ণ ডেটা মুছে ফেলা হয়েছে

📌 5. Privacy Issue

ব্যক্তি নিজের তথ্য দিতে চায়নি

👉 উদাহরণ: income, phone number

📌 6. Not Applicable

সব attribute সব record-এর জন্য প্রযোজ্য না

👉 উদাহরণ: "spouse name" (unmarried মানুষের জন্য নেই)

? Why Noisy? (ডেটা ভুল বা noisy কেন হয়?)

1. Measurement Error

যন্ত্র সঠিক নয়

👉 উদাহরণ: stopwatch দিয়ে wind speed মাপা ❌

2. Human Error

মানুষ ভুল করে data entry করেছে

👉 উদাহরণ: 100 এর জায়গায় 1000 লিখেছে

3. Computer Error

software bug বা system error

4. Transmission Error

network error এর কারণে data corrupted হয়েছে

5. Deliberate Error

ইচ্ছাকৃতভাবে ভুল data দেওয়া

👉 উদাহরণ: fake age

6. Data Decay

সময় পার হওয়ার সাথে সাথে data পুরানো হয়ে যায়

👉 উদাহরণ: address change

? Why Inconsistent? (ডেটা অসামঞ্জস্যপূর্ণ কেন হয়?)

1. Data Integration Problem

ভিন্ন database একসাথে করার সময় সমস্যা হয়

2. Different Naming

একই attribute-এর ভিন্ন নাম

👉 উদাহরণ:

- customer
- cust
- cust_id

3. Different Units

একই মান ভিন্ন এককে

👉 উদাহরণ:

- 1.75 m
- 175 cm

👉 এগুলো convert না করলে mismatch হবে

Data Quality (ডেটার গুণগত মান) — Example সহ বিস্তারিত ব্যাখ্যা ACCTBI

ডেটার মান (quality) বোঝার জন্য আমরা কিছু গুরুত্বপূর্ণ মাপকাঠি (measures) ব্যবহার করি। এখন প্রতিটি বিষয় বাস্তব উদাহরণসহ (real-life example) বিস্তারিতভাবে বোঝানো হলো

1. Accuracy (সঠিকতা)

 সংজ্ঞা:

ডেটা আসলেই বাস্তবতার সাথে মিলছে কিনা, অর্থাৎ ডেটা সঠিক (correct) কিনা — সেটাই Accuracy।

 উদাহরণ:

👉 Example 1:

- age = 25  (সঠিক)
- age = -5  (অসম্ভব, তাই ভুল)

👉 Example 2:

- Salary = 50,000 টাকা (যদি সত্যি হয়) 
 - Salary = 5,00,000 টাকা (ভুলভাবে entry হয়েছে) 
-

2. Completeness (সম্পূর্ণতা)

 সংজ্ঞা:

ডেটাতে সব প্রয়োজনীয় attribute আছে কিনা, অর্থাৎ কিছু missing আছে কিনা

 উদাহরণ:

👉 Example 1:

Name	Age	Email
Rahim	25	rahim@gmail.com
Karim	NULL ❌	karim@gmail.com

👉 Karim-এর age missing → incomplete data

👉 Example 2:

- customer database-এ phone number নাই ❌
- address নাই ❌

👉 গুরুত্বপূর্ণ তথ্য না থাকলে analysis অসম্পূর্ণ হবে

✅ 3. Consistency (সামঞ্জস্যতা)

🔍 সংজ্ঞা:

ডেটা একে অপরের সাথে মিলছে কিনা, অর্থাৎ **contradiction** আছে কিনা

📌 উদাহরণ:

👉 Example 1:

- Age = 20
- Birthday = 1990 ❌

👉 1990 হলে age 20 হতে পারে না → inconsistent

👉 Example 2:

- Height = 1.75 m
- Height = 175 cm ✅ (same, consistent)

👉 কিন্তু যদি:

- Height = 1.75 m
 - Height = 200 cm ❌ (mismatch)
-

👉 Example 3 (Duplicate Record):

ID	Name	Age
----	------	-----

101	Rahim	25
-----	-------	----

101	Rahim	30 ❌
-----	-------	------

👉 একই ID, কিন্তু age আলাদা → inconsistent

✅ 4. Timeliness (সময়োপযোগিতা)

🔍 সংজ্ঞা:

ডেটা আপডেটেড (up-to-date) কিনা

📌 উদাহরণ:

👉 Example 1:

- Address: Dhaka (2020 সালে)
- কিন্তু এখন (2026) → Rajshahi ❌

👉 পুরানো data → ভুল সিদ্ধান্ত হতে পারে

👉 Example 2:

- Stock price = 100 (গত সপ্তাহের) ❌
- কিন্তু আজ price = 120

👉 পুরানো data ব্যবহার করলে analysis ভুল হবে

✓ 5. Believability (বিশ্বাসযোগ্যতা)

🔍 সংজ্ঞা:

ডেটা কতটা বিশ্বাসযোগ্য (trustworthy)

📌 উদাহরণ:

👉 Example 1:

- Government database → বেশি reliable ✓
 - Random Facebook post → কম reliable ✗
-

👉 Example 2:

- User নিজের income লিখেছে: 10,00,000 টাকা ✗ (fake হতে পারে)
 - Official salary record → বেশি বিশ্বাসযোগ্য ✓
-

👉 Example 3:

- Scientific sensor data (verified) → believable ✓
 - manually guessed data → believable না ✗
-

✓ 6. Interpretability (বোঝার সহজতা)

🔍 সংজ্ঞা:

ডেটা কত সহজে বোঝা যায়, format clear কিনা

📌 উদাহরণ:

👉 Example 1:

- Date: 03/04/2024 ✗ (confusing → March না April?)
 - Date: 2024-04-03 ✓ (clear format)
-

👉 Example 2:

- Salary: 50000 ✗ (currency unclear)
- Salary: 50000 BDT ✓ (clear)

👉 Example 3:

- Gender: M/F ❌ (confusing for some systems)
 - Gender: Male/Female ✅ (clear)
-

📖 Why is Data Preprocessing Important? (ডেটা প্রিপ্রসেসিং কেন গুরুত্বপূর্ণ?)

📖 ভূমিকা (Introduction)

ডেটা মাইনিং (Data Mining) বা মেশিন লার্নিং (Machine Learning)-এর সফলতা সম্পূর্ণভাবে নির্ভর করে ডেটার গুণগত মানের উপর। যদি ডেটা ভালো না হয়, তাহলে যত উন্নত algorithm-ই ব্যবহার করা হোক না কেন, ফলাফল কখনোই নির্ভুল হবে না।

👉 তাই একটি গুরুত্বপূর্ণ নীতি হলো:

💡 **“No quality data → No quality mining results”**

অর্থাৎ, ভালো ডেটা না থাকলে ভালো ফলাফল পাওয়া সম্ভব না।

? How about mining data directly? (প্রিপ্রসেসিং ছাড়া সরাসরি ডেটা ব্যবহার করলে কী হয়?)

অনেক সময় মনে হতে পারে, “ডেটা আছে, সরাসরি algorithm চালালেই তো হয়!” — কিন্তু বাস্তবে এটি বড় ভুল ধারণা।

⚠️ সমস্যাগুলো:

🔴 1. Mining Algorithm সবসময় শক্তিশালী না (Not Robust)

যদিও কিছু algorithm incomplete বা noisy data handle করতে পারে, কিন্তু:

- সব algorithm তা পারে না ❌
- অনেক ক্ষেত্রে ভুল ফলাফল দেয় ❌

📌 উদাহরণ:

- যদি Salary = -10 থাকে → model ভুল prediction করবে
-

🔴 2. Inconsistency handle করতে পারে না

বেশিরভাগ mining algorithm **inconsistent data** handle করতে পারে না

📌 উদাহরণ:

- Age = 25
- Birthday = 1990 ❌

👉 algorithm confuse হয়ে যাবে

🔴 3. Data Size খুব বড় হতে পারে

ডেটা অনেক বড় (huge volume) হলে:

- processing slow হয়ে যায়
- memory problem হয়
- computation cost বেড়ে যায়

📌 উদাহরণ:

- 1 million record → সরাসরি process করা কঠিন
-

🔴 4. Hidden Problem থাকে

Even যদি:

- incomplete না হয়
- noisy না হয়
- inconsistent না হয়

তবুও data properly structured না হলে analysis ঠিকমতো হবে না।

Key Insight (গুরুত্বপূর্ণ তথ্য)

👉 বাস্তবে,

📌 **Data Preprocessing** প্রায় 70% কাজের অংশ দখল করে

অর্থাৎ, পুরো data mining project-এর সবচেয়ে বড় সময় ব্যয় হয় data প্রস্তুত করতে।

Major Tasks in Data Preprocessing (ডেটা প্রিপ্রেসেসিং-এর প্রধান কাজসমূহ)

এখন আমরা preprocessing-এর মূল ধাপগুলো বিস্তারিতভাবে দেখবো 📌

🔵 1. Data Cleaning (ডেটা পরিষ্কার করা)

🔍 সংজ্ঞা:

ডেটার ভুল, missing, এবং অপ্রয়োজনীয় অংশগুলো ঠিক করা বা সরানো

🔧 কী কী করা হয়:

- missing value fill করা
- noisy data smooth করা
- outlier identify/remove করা
- inconsistency ঠিক করা

📌 উদাহরণ:

👉 Before Cleaning:

Name	Age	Salary
Rahim	25	50000
Karim	NULL	-100 ❌

👉 After Cleaning:

Name	Age	Salary
Rahim	25	50000
Karim	30 (estimated)	20000

🔵 2. Data Integration (ডেটা একত্র করা)

🔍 সংজ্ঞা:

ভিন্ন ভিন্ন source (database, file) থেকে ডেটা একত্র করা

📌 উদাহরণ:

👉 Database 1:

ID	Name
----	------

101	Rahim
-----	-------

👉 Database 2:

ID	Salary
----	--------

101	50000
-----	-------

👉 After Integration:

ID	Name	Salary
----	------	--------

101	Rahim	50000
-----	-------	-------

⚠️ এখানে সমস্যা হতে পারে:

- duplicate data
 - different naming (cust vs customer)
-

3. Data Reduction (ডেটা কমানো)

 সংজ্ঞা:

ডেটার size কমানো, কিন্তু মূল তথ্য ঠিক রাখা

 লক্ষ্য:

👉 কম data → faster processing → same result

 উদাহরণ:

👉 Original Data:

- 10,00,000 rows

👉 Reduced Data:

- 1,00,000 rows (sampling)

👉 Result প্রায় একই, কিন্তু processing faster

Techniques:

- Sampling
 - Aggregation
 - Dimensionality reduction
-

4. Data Transformation & Data Discretization

সংজ্ঞা:

ডেটাকে এমনভাবে পরিবর্তন করা যাতে analysis সহজ হয়

♦ (a) Normalization (স্কেল ঠিক করা)

👉 সব value একই range-এ আনা

📌 উদাহরণ:

- Salary: 10000 – 100000
- Age: 20 – 60

👉 normalize করে 0–1 range-এ আনা হয়

♦ (b) Aggregation (সমষ্টি করা)

👉 ছোট ছোট data একত্র করা

📌 উদাহরণ:

- Daily sales → Monthly sales
-

♦ (c) Discretization (continuous → discrete)

👉 continuous value কে category-তে ভাগ করা

📌 উদাহরণ:

- Age = 23 → “Young”
 - Age = 45 → “Middle-aged”
-

Final Summary (সংক্ষেপে)

কেন গুরুত্বপূর্ণ?

- ভুল data → ভুল result
 - algorithm সব problem handle করতে পারে না
 - large data → slow processing
-

Major Tasks:

1. Data Cleaning
 2. Data Integration
 3. Data Reduction
 4. Data Transformation & Discretization
-

শেষ কথা (Exam Ready Line)



“Data preprocessing is a crucial step in data mining because it improves data quality, reduces complexity, and ensures accurate and efficient analysis.”

Data Cleaning (ডেটা ক্লিনিং)

ভূমিকা (Introduction)

ডেটা প্রিপ্রসেসিং-এর সবচেয়ে গুরুত্বপূর্ণ ধাপগুলোর একটি হলো **Data Cleaning** (ডেটা পরিষ্কার করা)। বাস্তব জীবনের ডেটা সাধারণত অসম্পূর্ণ (missing), ভুল (noisy), এবং অসামঞ্জস্যপূর্ণ (inconsistent) থাকে। এই ধাপে আমরা সেইসব সমস্যা ঠিক করি যাতে ডেটা analysis-এর জন্য উপযোগী হয়।

Data Cleaning-এর মূল কাজ (Main Tasks)

1. **Fill in Missing Values** (অনুপস্থিত ডেটা পূরণ করা)
2. **Identify Outliers & Smooth Noisy Data** (ভুল/অস্বাভাবিক ডেটা ঠিক করা)
3. **Correct Inconsistent Data** (অসামঞ্জস্যপূর্ণ ডেটা ঠিক করা)

◆ 1. How to Handle Missing Data? (Missing Value কীভাবে Handle করা হয়?)

● Method 1: Ignore the Tuple (পুরো row বাদ দেওয়া)

👉 যখন কোনো গুরুত্বপূর্ণ attribute (যেমন class label) missing থাকে, তখন পুরো record delete করা হয়।

📌 উদাহরণ:

Name	Age	Result
------	-----	--------

Rahim	25	Pass
-------	----	------

Karim	22	NULL ❌
-------	----	--------

👉 Karim-এর Result নেই → পুরো row delete করা হতে পারে

⚠️ Problem:

- যদি অনেক data delete করা হয় → dataset ছোট হয়ে যাবে
- imbalance তৈরি হতে পারে

● Method 2: Manual Filling (হাতে পূরণ করা)

👉 মানুষ নিজে বসে missing value পূরণ করে

📌 উদাহরণ:

- Karim-এর age manually 23 দেওয়া হলো

⚠️ Problem:

- সময়সাপেক্ষ (tedious)
 - বড় dataset-এর জন্য অসম্ভব (infeasible)
-

🔴 Method 3: Global Constant দিয়ে পূরণ

👉 সব missing value-এ একটি constant বসানো হয়

📌 উদাহরণ:

- Age = "Unknown"
- Salary = 0

⚠️ Problem:

- এটি নতুন category তৈরি করে
 - model confuse হতে পারে
-

🔴 Method 4: Attribute Mean ব্যবহার করা

👉 missing value → average দিয়ে replace করা

📌 উদাহরণ:

Age values: 20, 25, 30

👉 Mean = $(20+25+30)/3 = 25$

👉 missing age → 25 বসানো হলো

🔴 Method 5: Class-wise Mean (Smarter Way)

👉 একই class-এর data থেকে mean নেওয়া হয়

📌 উদাহরণ:

Name	Age	Class
A	20	Student
B	22	Student

C NULL Studen
t

👉 Mean = $(20+22)/2 = 21$

👉 Missing value = 21

✓ এটি বেশি accurate

🔴 Method 6: Most Probable Value (Inference-based)

👉 advanced technique ব্যবহার করা হয়
যেমন:

- Bayesian Formula
- Decision Tree

📌 উদাহরণ:

- যদি income, education দেখে age predict করা হয়

👉 এটি সবচেয়ে intelligent method

🔹 2. How to Handle Noisy Data? (Noisy Data কীভাবে ঠিক করা হয়?)

🔵 Method 1: Binning (ডেটাকে গ্রুপে ভাগ করা)

ছবির উদাহরণের মাধ্যমে **Equi-width Binning** এবং **Bin Boundaries** দিয়ে ডেটা স্মুথিং (Smoothing) করার প্রক্রিয়াটি নিচে সহজভাবে বুঝিয়ে বলা হলো:

১. Equi-width Binning (সমান প্রস্থের বিন)

এখানে ডেটার রেঞ্জকে (সর্বোচ্চ মান - সর্বনিম্ন মান) সমান কয়েকটি ভাগে ভাগ করা হয়। ছবিতে ৩টি বিন করা হয়েছে:

- নিয়ম: $(\text{সর্বোচ্চ } ৩৪ - \text{সর্বনিম্ন } ৪) \div ৩ = ১০।$ অর্থাৎ প্রতিটি বিনের প্রস্থ হবে ১০।
- **Bin 1** [৪ থেকে ১৩]: এই সীমার মধ্যে থাকা সংখ্যাগুলো হলো ৪, ৮।

- **Bin 2** [১৪ থেকে ২৩]: এই সীমার মধ্যে থাকা সংখ্যাগুলো হলো ১৫, ২১, ২১।
 - **Bin 3** [২৪ থেকে ৩৪]: এই সীমার মধ্যে থাকা সংখ্যাগুলো হলো ২৪, ২৫, ২৮, ৩৪।
-

২. Smoothing by Bin Boundaries (সীমানার মাধ্যমে মসৃণকরণ)

এই পদ্ধতিতে একটি বিনের প্রতিটি সংখ্যাকে ওই বিনের সবচেয়ে কাছের সীমানা (**Boundary**) মান দিয়ে পরিবর্তন করা হয়। প্রতিটি বিনের দুটি সীমানা থাকে: একটি সর্বনিম্ন (**Minimum**) এবং একটি সর্বোচ্চ (**Maximum**)।

ছবির উদাহরণ অনুযায়ী পরিবর্তনগুলো লক্ষ্য করুন:

- **Bin 1** (৪, ৮): সীমানা হলো ৪ এবং ৮। এখানে সংখ্যাগুলো নিজে নিজেই সীমানা, তাই কোনো পরিবর্তন হয়নি। (রেজাল্ট: ৪, ৮)
- **Bin 2** (১৫, ২১, ২১): সীমানা হলো ১৫ এবং ২১।
 - ১৫ নিজে একটি সীমানা।
 - ২১ নিজে একটি সীমানা। (রেজাল্ট: ১৫, ২১, ২১)
- **Bin 3** (২৪, ২৫, ২৮, ৩৪): সীমানা হলো ২৪ এবং ৩৪।
 - ২৪ নিজে একটি সীমানা।
 - ২৫ সংখ্যাটি ২৪ এর বেশি কাছে, তাই এটি ২৪ হয়ে যাবে।
 - ২৮ সংখ্যাটি ৩৪ এর বেশি কাছে (দূরত্ব ৬), যেখানে ২৪ থেকে দূরত্ব বেশি (দূরত্ব ১০), তাই এটি ৩৪ হয়ে যাবে।
 - ৩৪ নিজে একটি সীমানা। (রেজাল্ট: ২৪, ২৫, ২৮, ৩৪ এর বদলে ছবিতে ২৪, ২৫, ২৫, ৩৪ দেখানো হয়েছে যৌক্তিক বিচারে)।

সহজ কথা: বিনের ভেতরের সংখ্যাটি যদি ছোট সীমানার কাছে হয় তবে ছোটটি বসবে, আর বড় সীমানার কাছে হলে বড়টি বসবে।

আপনি কি **Equi-depth binning** বা এর **Mean/Median smoothing** সম্পর্কে আরও বিস্তারিত জানতে চান?

● Method 2: Clustering (গ্রুপ তৈরি করা)

👉 similar data একসাথে group করা হয়

📌 উদাহরণ:

- Cluster 1: (20, 21, 22)
- Cluster 2: (50, 52, 55)

👉 যদি value = 200 ❌

👉 এটি কোনো cluster-এ পড়ে না → outlier

✓ outlier remove করা হয়

● Method 3: Regression (গাণিতিক fitting)

👉 একটি function দিয়ে data fit করা হয়

📌 উদাহরণ:

- Salary vs Experience

👉 Expected line:

- Experience বাড়লে salary বাড়বে

👉 যদি:

- Experience = 10 বছর
- Salary = 5000 ❌

👉 এটি expected line থেকে অনেক দূরে → outlier

◆ 3. How to Handle Inconsistent Data?

🔍 সংজ্ঞা:

যখন data conflict করে বা rules ভাঙে

● Method 1: Manual Correction (হাতে ঠিক করা)

📌 উদাহরণ:

- Age = 25
- DOB = 1990 ❌

👉 manually ঠিক করা

👉 paper trace করে error খুঁজে বের করা হয়

● Method 2: External Reference ব্যবহার

👉 trusted source দিয়ে verify করা

📌 উদাহরণ:

- Official database check করা
-

Method 3: Constraint Checking (Rule-based)

👉 predefined rule ব্যবহার করা

📌 উদাহরণ:

- Age ≥ 0
- Salary ≥ 0

👉 rule violate করলে error detect

Method 4: Knowledge Engineering Tools

👉 system automatically inconsistency detect করে

⚠️ **Important Note:**

👉 Inconsistent data ঠিক করতে প্রায়ই **human intervention** (মানুষের সাহায্য) লাগে

Final Summary (মনে রাখার জন্য)

🔑 **Missing Data Handling:**

- Ignore tuple
 - Manual fill
 - Global constant
 - Mean / Class mean
 - Prediction (AI-based)
-

🔑 **Noisy Data Handling:**

- Binning
 - Clustering
 - Regression
-

Inconsistent Data Handling:

- Manual correction
- External source
- Rule checking

Exam Ready Conclusion



“Data cleaning improves data quality by handling missing values, smoothing noisy data, and resolving inconsistencies, which ensures accurate and reliable data mining results.”

Data Integration (ডেটা ইন্টিগ্রেশন)

Data Preprocessing-এর একটি অত্যন্ত গুরুত্বপূর্ণ ধাপ হলো **Data Integration**। সহজ কথায়, যখন আমরা ভিন্ন ভিন্ন উৎস (Sources) থেকে ডেটা সংগ্রহ করে একটি নির্দিষ্ট জায়গায় (যেমন: Data Warehouse) জমা করি, তখন তাকেই Data Integration বলে।

নিচে বইয়ের ভাষায় সহজ উদাহরণসহ বিস্তারিত আলোচনা করা হলো:

১. Schema Integration (স্কিমা ইন্টিগ্রেশন)

বিভিন্ন সোর্স থেকে আসা ডেটার গঠন বা স্ট্রাকচার আলাদা হতে পারে। সেগুলোকে এক সূতায় গাঁথাই হলো স্কিমা ইন্টিগ্রেশন। এর প্রধান দুটি সমস্যা হলো:

- **Entity Identification Problem:** ভিন্ন ভিন্ন ডেটাবেজে একই জিনিসকে আলাদা নামে ডাকা হতে পারে।
 - উদাহরণ: একটি ডেটাবেজে একজনের নাম লেখা আছে **Family Name** হিসেবে, অন্যটিতে লেখা আছে **Surname** হিসেবে। সিস্টেমকে বুঝতে হবে যে এই দুটো আসলে একই জিনিস।
- **Semantic Heterogeneity** (সেম্যান্টিক ভিন্নতা): একই জিনিসের ভ্যালু বা মান ভিন্ন ভিন্ন এককে (Unit) থাকতে পারে।
 - উদাহরণ: একজন ব্যক্তির উচ্চতা এক জায়গায় লেখা **175cm**, অন্য জায়গায় লেখা **1.75m**। ইন্টিগ্রেশনের সময় এগুলোকে একটি নির্দিষ্ট স্কেলে নিয়ে আসতে হয়।

নোট: এই সমস্যাগুলো সমাধানের জন্য **Metadata** (ডেটা সম্পর্কে তথ্য) ব্যবহার করা হয়, যা ডেটার অর্থ বুঝতে সাহায্য করে।

২. Handling Redundancy (অতিরিক্ত বা অনাবশ্যক ডেটা নিয়ন্ত্রণ)

ডেটা ইন্টিগ্রেশন করার সময় প্রায়ই **Redundancy** বা তথ্যের পুনরাবৃত্তি ঘটে। এর প্রধান কারণগুলো হলো:

- **Object Identification:** একই বস্তুর ভিন্ন ভিন্ন নাম থাকা। (যেমন: `Customer_ID` এবং `Client_Number` একই ব্যক্তির জন্য ব্যবহৃত হওয়া)।
- **Derivable Data (উদ্ভূত ডেটা):** একটি টেবিলের তথ্য যদি অন্য টেবিলের তথ্যের ওপর ভিত্তি করে তৈরি করা যায়, তবে তাকে রিডান্ডেন্ট বলা হয়।
 - উদাহরণ: একটি টেবিলে `Monthly Salary` আছে, অন্য টেবিলে `Annual Revenue` রাখা হয়েছে। যেহেতু মাসের বেতন দিয়ে বছরের আয় বের করা সম্ভব, তাই বছরের আয়ের কলামটি এখানে বাড়তি বা রিডান্ডেন্ট।
- **Duplicate Tuples:** অনেক সময় একই তথ্য বা সারি (Row) বারবার চলে আসে।

৩. Correlation and Covariance Analysis (সহসম্বন্ধ বিশ্লেষণ)

অপ্রয়োজনীয় বা রিডান্ডেন্ট ডেটা শনাক্ত করার জন্য গণিত ও পরিসংখ্যানের দুটি পদ্ধতি ব্যবহার করা হয়:

1. **Correlation Analysis (কো-রিলেশন):** যদি একটি অ্যাট্রিবিউটের মান বাড়লে অন্যটির মানও বাড়ে (যেমন: শিক্ষা বাড়লে আয় বাড়ে), তবে তাদের মধ্যে কো-রিলেশন আছে বলা হয়। যদি দুটি অ্যাট্রিবিউট একে অপরের ওপর শতভাগ নির্ভরশীল হয়, তবে একটিকে বাদ দেওয়া যায়।
2. **Covariance Analysis (কো-ভ্যারিয়েন্স):** এটিও দুটি ভেরিয়েবলের মধ্যে সম্পর্ক বা নির্ভরশীলতা যাচাই করতে ব্যবহৃত হয়।

৪. কেন Data Integration জরুরি?

সঠিকভাবে ডেটা ইন্টিগ্রেট করলে নিচের সুবিধাগুলো পাওয়া যায়:

- **Redundancy** হ্রাস পায়: ফালতু ডেটা কমে যায়।
- **Inconsistencies** দূর হয়: তথ্যের অসঙ্গতি বা অমিল থাকে না।
- **Mining Quality & Speed:** ডেটা মাইনিং বা বিশ্লেষণের গতি বেড়ে যায় এবং রেজাল্ট অনেক বেশি নির্ভুল হয়।

সহজ কথায়, **Data Integration** হলো অনেকগুলো এলোমেলো তথ্যের ভাগ্যকে সাজিয়ে গুছিয়ে একটি পরিষ্কার এবং কাজের উপযোগী ডেটাসেট তৈরি করা।

- ✓ 1. **Redundancy** কমে
- ✓ 2. **Inconsistency** কমে
- ✓ 3. **Data quality improve** হয়
- ✓ 4. **Mining speed** বাড়ে
- ✓ 5. **Accurate result** পাওয়া যায়

Final Summary (সংক্ষেপে)

🔑 Data Integration কী?

👉 বিভিন্ন source থেকে data combine করা

🔑 Main Problems:

1. Entity Identification Problem
 2. Semantic Heterogeneity
 3. Redundancy (duplicate, derived data)
-

🔑 Solutions:

- Schema Integration
 - Metadata ব্যবহার
 - Correlation Analysis
 - Careful data merging
-

🎯 Exam Ready Conclusion



“Data integration combines data from multiple sources into a unified dataset while resolving schema conflicts, semantic differences, and redundancy, thereby improving data quality and mining efficiency.”

ডেটা ইন্টিগ্রেশনের সময় রিডান্ড্যান্সি (Redundancy) চেক করার জন্য

Correlation

Analysis

বা সহসম্বন্ধ বিশ্লেষণ খুবই গুরুত্বপূর্ণ। এটি মূলত দুই ধরনের ডেটার জন্য দুইভাবে করা হয়: **Nominal** (নামবাচক) এবং **Numeric** (সংখ্যাবাচক)।

নিচে সহজ ব্যাখ্যা দেওয়া হলো:

১. Nominal Data-র জন্য: χ^2 (Chi-square) Test

যখন ডেটাগুলো ক্যাটাগরি বা নাম আকারে থাকে (যেমন: জন্মদার, পেশা), তখন আমরা Chi-square টেস্ট করি।

- মূল কথা: χ^2 এর মান যত বেশি হবে, ভ্যারিয়েবল দুটির মধ্যে সম্পর্ক (Correlation) তত বেশি শক্তিশালী হবে।
- **Actual vs Expected Count:** আমরা বাস্তবে যে ডেটা পেয়েছি (Actual) এবং গাণিতিকভাবে যা পাওয়ার কথা ছিল (Expected)—এই দুইয়ের মধ্যে পার্থক্য যত বেশি হবে, χ^2 এর মান তত বাড়বে।

- **Causality** (কারণ সম্পর্ক): একটি গুরুত্বপূর্ণ কথা মনে রাখতে হবে—**Correlation** মানেই **Causality** নয়।
 - উদাহরণ: একটি শহরে হাসপাতালের সংখ্যা বাড়লে গাড়ী চুরির সংখ্যাও বাড়তে দেখা যেতে পারে। তার মানে এই নয় যে হাসপাতাল গাড়ী চুরি বাড়চ্ছে। আসলে "জনসংখ্যা" (Population) বাড়লে এই দুটোই বাড়ে। এখানে জনসংখ্যা হলো আসল কারণ।

- **X² (chi-square) test**

$$\chi^2 = \sum \frac{(\text{Observed} - \text{Expected})^2}{\text{Expected}}$$

- The larger the X² value, the more likely the variables are related
- The cells that contribute the most to the X² value are those whose actual count is very different from the expected count
- Correlation does not imply causality
 - # of hospitals and # of car-theft in a city are correlated
 - Both are causally linked to the third variable: population

Chi-Square Calculation: An Example

	Play chess	Not play chess	Sum (row)
Like science fiction	250(90)	200(360)	450
Not like science fiction	50(210)	1000(840)	1050
Sum(col.)	300	1200	1500

- X² (chi-square) calculation (numbers in parenthesis are expected counts calculated based on the data distribution in the two categories)

$$\chi^2 = \frac{(250 - 90)^2}{90} + \frac{(50 - 210)^2}{210} + \frac{(200 - 360)^2}{360} + \frac{(1000 - 840)^2}{840} = 507.93$$

- It shows that two attributes are correlated in the group

যখন ডেটাগুলো সংখ্যায় থাকে (যেমন: বয়স, বেতন), তখন আমরা **Pearson's product moment coefficient** ব্যবহার করি। এর মান -1 থেকে $+1$ এর মধ্যে থাকে।

- **Positive Correlation ($r > 0$):** যদি একটির মান বাড়লে অন্যটির মানও বাড়ে। যেমন: পড়াশোনার সময় বাড়লে পরীক্ষার রেজাল্ট ভালো হয়।
- **Negative Correlation ($r < 0$):** যদি একটির মান বাড়লে অন্যটির মান কমে। যেমন: গাড়ির গতি বাড়লে গন্তব্যে পৌঁছানোর সময় কমে।
- **Independent ($r = 0$):** যখন একটির সাথে অন্যটির কোনো সম্পর্কই থাকে না।

$$r = \frac{n \sum xy - (\sum x)(\sum y)}{\sqrt{[n \sum x^2 - (\sum x)^2][n \sum y^2 - (\sum y)^2]}}$$

$$r = \frac{\sum xy - \frac{(\sum x)(\sum y)}{n}}{\sqrt{[\sum x^2 - \frac{(\sum x)^2}{n}][\sum y^2 - \frac{(\sum y)^2}{n}]}}$$

৩. Scatter Plots (ছক কাগজ বা বিচ্ছুরণ চিত্র)

ভিজুয়ালি বা দেখে চেনার সবচেয়ে সহজ উপায় হলো **Scatter Plot**। ডেটা পয়েন্টগুলো যদি একটি লাইনের মতো উপরের দিকে যায়, তবে সেটা পজিটিভ; নিচের দিকে নামলে নেগেটিভ। আর যদি পয়েন্টগুলো চারদিকে ছিটিয়ে থাকে, তবে বুঝতে হবে কোনো সম্পর্ক নেই।

বইয়ের ভাষায় সারাংশ:

ডেটা ইন্টিগ্রেট করার সময় যদি দেখা যায় দুটি অ্যাট্রিবিউট (যেমন: 'Income' এবং 'Tax') একে অপরের সাথে স্ট্রংলি কো-রিলেটেড, তবে ডেটাসেট হালকা করার জন্য আমরা একটি অ্যাট্রিবিউট বাদ দিতে পারি। এটি মডেলের গতি বাড়াতে সাহায্য করে।

তোমার কি χ^2 ক্যালকুলেশন বা Pearson's সূত্রটি বিস্তারিত জানার প্রয়োজন আছে?

Covariance

হলো এমন একটি পরিসংখ্যানগত পরিমাপ যা নির্দেশ করে দুটি ভ্যারিয়েবল বা অ্যাট্রিবিউট একে অপরের সাথে কীভাবে পরিবর্তিত হয়। এটি অনেকটা Correlation-এর মতোই, তবে এটি সরাসরি ভ্যারিয়েবলগুলোর স্কেলের ওপর নির্ভর করে।

নিচে বইয়ের ভাষায় এবং তোমার দেওয়া উদাহরণের মাধ্যমে বিস্তারিত বুঝিয়ে দিচ্ছি:

১. Covariance-এর প্রকৃতি

কো-ভ্যারিয়েন্সের মান দেখে আমরা বুঝতে পারি দুটি ভ্যারিয়েবল কি একই দিকে যাচ্ছে নাকি বিপরীত দিকে:

- **Positive Covariance (\$Cov(A, B) > 0\$):** যদি একটি ভ্যারিয়েবল তার গড় মানের (Mean/Expected value) চেয়ে বেশি হয়, তবে অন্যটিও সাধারণত তার গড় মানের চেয়ে বেশি হবে। অর্থাৎ, তারা একই দিকে পরিবর্তিত হয় (দুটিই বাড়ে বা দুটিই কমে)।
- **Negative Covariance (\$Cov(A, B) < 0\$):** যদি একটি ভ্যারিয়েবল তার গড় মানের চেয়ে বেশি হয়, তবে অন্যটি তার গড় মানের চেয়ে কম হওয়ার প্রবণতা দেখায়। অর্থাৎ, তারা বিপরীত দিকে কাজ করে।
- **Independence (\$Cov(A, B) = 0\$):** যদি ভ্যারিয়েবল দুটি সম্পূর্ণ স্বাধীন হয়, তবে কো-ভ্যারিয়েন্স শূন্য হবে। তবে মনে রাখবে, কো-ভ্যারিয়েন্স শূন্য হলেই যে তারা সবসময় স্বাধীন হবে—এমনটা নাও হতে পারে (Converse is not true)।
 - Correlation coefficient (also called **Pearson's product moment coefficient**)

$$r_{A,B} = \frac{\sum_{i=1}^n (a_i - \bar{A})(b_i - \bar{B})}{n\sigma_A\sigma_B} = \frac{\sum_{i=1}^n (a_i b_i) - n\bar{A}\bar{B}}{n\sigma_A\sigma_B}$$

where n is the number of tuples, $\bar{A}$ and $\bar{B}$ are the respective means of A and B , σ_A and σ_B are the respective standard deviation of A and B , and $\sum(a_i b_i)$ is the sum of the AB cross-product.

- If $r_{A,B} > 0$, A and B are positively correlated (A 's values increase as B 's). The higher, the stronger correlation.
- $r_{A,B} = 0$: independent; $r_{AB} < 0$: negatively correlated

Population Covariance Formula

$$Cov(x, y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N}$$

Sample Covariance

$$Cov(x, y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N-1}$$

২. উদাহরণের বিশ্লেষণ (Stock Prices)

তুমি যে উদাহরণটি দিয়েছ, সেটি দিয়ে বিষয়টি পরিষ্কার করা যাক। এখানে দুটি স্টকের দাম দেওয়া আছে:

$$A = \{2, 3, 5, 4, 6\} \text{ এবং } B = \{5, 8, 10, 11, 14\}$$

ধাপ ১: গড় মান বা **Expected Value** বের করা

$$E(A) = \frac{2 + 3 + 5 + 4 + 6}{5} = 4$$

$$E(B) = \frac{5 + 8 + 10 + 11 + 14}{5} = 9.6$$

ধাপ ২: কো-ভ্যারিয়েন্সের সূত্র প্রয়োগ

$$\text{সূত্রটি হলো: } \text{Cov}(A, B) = E(A \times B) - E(A) \times E(B)$$

$$E(A \times B) = \frac{(2 \times 5) + (3 \times 8) + (5 \times 10) + (4 \times 11) + (6 \times 14)}{5} = \frac{10 + 24 + 50 + 44 + 84}{5} = \frac{212}{5} = 42.4$$

$$\text{এখন, } \text{Cov}(A, B) = 42.4 - (4 \times 9.6) = 42.4 - 38.4 = 4$$

ফলাফল: যেহেতু $\text{Cov}(A, B) = 4$, যা শূন্যের চেয়ে বড় (> 0), তাই আমরা বলতে পারি স্টক A এবং B একসাথেই বাড়বে বা কমবে (Rise or fall together)।

Co-Variance: An Example

$$\text{Cov}(A, B) = E((A - \bar{A})(B - \bar{B})) = \frac{\sum_{i=1}^n (a_i - \bar{A})(b_i - \bar{B})}{n}$$

- It can be simplified in computation as

$$\text{Cov}(A, B) = E(A \cdot B) - \bar{A}\bar{B}$$

- Suppose two stocks A and B have the following values in one week: (2, 5), (3, 8), (5, 10), (4, 11), (6, 14).
- Question: If the stocks are affected by the same industry trends, will their prices rise or fall together?
 - $E(A) = (2 + 3 + 5 + 4 + 6) / 5 = 20 / 5 = 4$
 - $E(B) = (5 + 8 + 10 + 11 + 14) / 5 = 48 / 5 = 9.6$
 - $\text{Cov}(A, B) = (2 \times 5 + 3 \times 8 + 5 \times 10 + 4 \times 11 + 6 \times 14) / 5 - 4 \times 9.6 = 4$
- Thus, A and B rise together since $\text{Cov}(A, B) > 0$.

৩. Correlation বনাম Covariance

তোমার মনে প্রশ্ন আসতে পারে, দুইটাই তো একই জিনিস বোঝাচ্ছে, তাহলে পার্থক্য কী?

বৈশিষ্ট্য	Covariance	Correlation
মান (Range)	$-\infty$ থেকে $+\infty$ পর্যন্ত হতে পারে।	সবসময় -1 থেকে $+1$ এর মধ্যে থাকে।

ইউনিট	এটি ডেটার ইউনিটের ওপর নির্ভর করে (যেমন: টাকা $\times$ কেজি)।	এর কোনো ইউনিট নেই (Pure number)।
ব্যবহার	সম্পর্কের দিক (Direction) বোঝার জন্য ভালো।	সম্পর্কের দিক এবং শক্তি (Strength) দুটো বোঝার জন্যই সেরা।

উপসংহার:

ডেটা প্রি-প্রসেসিংয়ের সময় যদি আমরা দেখি দুটি কলামের মধ্যে স্ট্রং পজিটিভ কো-ভ্যারিয়েন্স আছে, তবে আমরা বুঝতে পারি তারা একে অপরের পরিপূরক। এটি আমাদের ডেটা ইন্টিগ্রেশনের সময় অপ্রয়োজনীয় বা ডুপ্লিকেট তথ্য বাদ দিতে সাহায্য করে।



Data Integration Hierarchical Structure

1. Schema Integration (Parent)

এটি হলো ডেটা ইন্টিগ্রেশনের প্রথম ধাপ যেখানে বিভিন্ন সোর্সের কাঠামো মেলানো হয়।

- **Child - Entity Identification Problem:** (একই জিনিস, ভিন্ন নাম। উদা: ID vs Number)
- **Child - Semantic Heterogeneity:** (একই জিনিস, ভিন্ন ইউনিট। উদা: cm vs m)

2. Handling Redundancy (Parent)

ইন্টিগ্রেশনের সময় বাড়তি বা ডুপ্লিকেট ডেটা কমানোর প্রক্রিয়া।

- **Child - Object Identification:** (একই অবজেক্টের আলাদা নাম শনাক্ত করা)
- **Child - Derivable/Derived Data:** (অন্য ডেটা থেকে যা বের করা যায়। উদা: Salary থেকে Annual Revenue)
- **Child - Duplicate Tuples:** (পুরো রো বা সারির পুনরাবৃত্তি)

3. Redundancy Detection Methods (Parent)

রিডান্ডেন্সি বা অতিরিক্ত ডেটা আছে কি না তা গাণিতিকভাবে ধরার জন্য এই পদ্ধতিগুলো ব্যবহার হয়।

- **Child A: Correlation Analysis (Nominal Data-র জন্য)**
 - Sub-child: χ^2 (Chi-square) Test
- **Child B: Correlation Analysis (Numeric Data-র জন্য)**
 - Sub-child: Pearson's Coefficient (r)
 - Sub-child: Scatter Plots (ভিজুয়াল রিলেশন দেখার জন্য)
- **Child C: Covariance Analysis (Numeric Data-র জন্য)**
 - Sub-child: $Cov(A, B)$ Calculation (গড় বা Expected Value-র ওপর ভিত্তি

করে)

4. Benefits of Data Integration (Conclusion/Outcome)

এটি কোনো টাইপ নয়, বরং উপরের সব কাজ করার পর যে ফলাফল পাওয়া যায়।

- Redundancy হ্রাস
- Inconsistency দূর
- Mining Speed & Quality বৃদ্ধি

Data Reduction Strategies

ডেটা রিডাকশন স্ট্রেটেজিগুলো ডেটা মাইনিংয়ের অত্যন্ত টেকনিক্যাল একটি অংশ। তুমি যে পয়েন্টগুলো উল্লেখ করেছ, সেগুলো সহজভাবে নিচে বুঝিয়ে দিচ্ছি:

১. Curse of Dimensionality (ডাইমেনশনালিটির অভিশাপ)

যখন আমরা বলি "High Dimensional Data", তার মানে হলো ডেটাসেটে কলাম বা ফিচারের সংখ্যা অনেক বেশি। এটি কেন সমস্যা তৈরি করে?

- **Data Sparsity:** ডাইমেনশন যত বাড়ে, ডেটা পয়েন্টগুলো একে অপরের থেকে তত দূরে সরে যায়। বিশাল মহাকাশে যেমন গ্রহগুলো অনেক দূরে দূরে থাকে, ডেটাও তেমন হয়ে যায়। একেই বলে **Sparsity**।
- **Distance becomes meaningless:** ক্লাস্টারিং বা আউটলিয়ার অ্যানালাইসিসে আমরা ডেটার দূরত্ব (Distance) মেপে কাজ করি। কিন্তু ডাইমেনশন খুব বেশি হলে সব পয়েন্টের দূরত্ব প্রায় সমান মনে হয়, ফলে ভালো রেজাল্ট পাওয়া অসম্ভব হয়ে পড়ে।
- **Exponential growth:** প্রতিটা নতুন কলাম যোগ হওয়ার সাথে সাথে সম্ভাব্য সাব-স্পেসের সংখ্যা জ্যামিতিক হারে (Exponentially) বাড়তে থাকে, যা প্রসেস করা কম্পিউটারের জন্য কঠিন হয়ে পড়ে।

২. Dimensionality Reduction (কেন দরকার?)

অপ্রয়োজনীয় ফিচার বাদ দিলে আমরা নিচের সুবিধাগুলো পাই:

- **Noise Removal:** অনেক সময় বাড়তি কলামে শুধু ভুল বা অপ্রাসঙ্গিক তথ্য থাকে, যা বাদ দিলে মডেল নির্ভুল হয়।
- **Performance:** মেমরি (Space) কম লাগে এবং অ্যালগরিদম দ্রুত (Time) কাজ শেষ করতে পারে।
- **Visualization:** আমরা সাধারণত ২ডি বা ৩ডি গ্রাফ বুঝতে পারি। ১০০টি কলামকে ২টিতে নামিয়ে আনলে ডেটা ভিজুয়ালাইজ করা সহজ হয়।

৩. Dimensionality Reduction Techniques

এই কমানোর কাজটি মূলত তিনভাবে করা হয়:

A. Wavelet Transforms

এটি মূলত সিগন্যাল প্রসেসিং বা ইমেজ ডেটার জন্য ব্যবহৃত হয়। এটি একটি ডেটা ভেক্টর $\$L\$$ -কে রূপান্তরিত করে অন্য একটি ভেক্টরে নিয়ে যায়। এতে ডেটার মূল কাঠামো ঠিক থাকে কিন্তু অপ্রয়োজনীয় ডিটেইলস বাদ দিয়ে আকার ছোট করা যায়।

B. Principal Component Analysis (PCA)

এটি একটি স্ট্যাটিস্টিক্যাল পদ্ধতি। মনে করো তোমার কাছে ১০টি কলাম আছে যেগুলো একে অপরের সাথে সম্পর্কিত। PCA এই ১০টি কলামকে গাণিতিকভাবে গুছিয়ে ২-৩টি নতুন কলামে (যাকে **Principal Components** বলে) রূপান্তর করে। এই নতুন কলামগুলোই মূল ডেটার প্রায় সব ইনফরমেশন বহন করে।

C. Feature Selection (Supervised & Nonlinear)

এটি হলো সেরা ফিচারের একটি সাবসেট খুঁজে বের করা।

- **Supervised:** এখানে আমরা দেখি কোন ফিচারগুলো আমাদের টার্গেট ভেরিয়েবল (যেমন: ক্যানসার হবে কি হবে না) প্রেডিক্ট করতে সবচেয়ে বেশি সাহায্য করছে।
- **Nonlinear:** অনেক সময় ফিচারের সাথে টার্গেট ভেরিয়েবলের সম্পর্ক সরলরেখায় থাকে না, তখন জটিল গাণিতিক মডেল দিয়ে ফিচার বাছাই করা হয়।

সংক্ষেপে:

ডাইমেনশনালিটি রিডাকশন হলো একটি বড় এবং ঝাপসা ছবিকে পিক্সেল কমিয়ে ছোট কিন্তু স্পষ্ট করা, যাতে কম্পিউটারের জন্য সেই ছবি চেনা সহজ হয়।

এই টেকনিকগুলোর মধ্যে **PCA** বা **Wavelet**-এর গাণিতিক ধাপগুলো কি তোমার সিলেবাসে আছে? থাকলে বলতে পারো, আমি উদাহরণ দিয়ে বুঝিয়ে দেব।

Data Reduction-এর প্রথম এবং প্রধান কৌশল হলো

Dimensionality

Reduction

এর মূল লক্ষ্য হলো ডেটাসেটের কলাম বা ফিচারের সংখ্যা কমানো, যাতে কাজ সহজ এবং দ্রুত হয়। নিচে তোমার দেওয়া পয়েন্টগুলো বইয়ের ভাষায় বিস্তারিত ব্যাখ্যা করা হলো:

১. কেন ডাইমেনশন কমাবো?

ডেটাসেটে দুই ধরনের সমস্যাযুক্ত অ্যাট্রিবিউট থাকতে পারে:

- **Redundant attributes** (অতিরিক্ত তথ্য): যখন একটি তথ্য অন্য কলাম থেকে সহজেই বোঝা যায়।
 - উদাহরণ: পণ্যের দাম (Purchase price) এবং তার ওপর দেওয়া সেলস ট্যাক্স (Sales tax)। যেহেতু দাম থেকেই ট্যাক্স বের করা যায়, তাই ট্যাক্সের আলাদা কলামটি রাখা অপ্রয়োজনীয়।
- **Irrelevant attributes** (অপ্রাসঙ্গিক তথ্য): যে তথ্যের সাথে আমাদের লক্ষ্যের কোনো সম্পর্ক নেই।
 - উদাহরণ: একজন ছাত্রের GPA প্রেডিক্ট করার জন্য তার 'Student ID' কোনো কাজে আসে না। এটি একটি অপ্রাসঙ্গিক তথ্য।

২. Attribute Subset Selection

এটি হলো একটি বড় কন্টেইনার থেকে শুধু দরকারি টুলগুলো বেছে নেওয়ার মতো। আমাদের লক্ষ্য থাকে এমন একটি **Minimum Set** খুঁজে বের করা যা ব্যবহার করলেও অরিজিনাল ডেটার মতো একই রেজাল্ট পাওয়া যায়।

- সমস্যা: যদি আমাদের কাছে d সংখ্যক অ্যাট্রিবিউট থাকে, তবে তাদের কম্বিনেশন হতে পারে 2^d টি। যদি ৫০টি কলাম থাকে, তবে 2^{50} টি কম্বিনেশন চেক করা কম্পিউটারের জন্য অসম্ভব।
- সমাধান: তাই আমরা **Heuristic Methods** (বুদ্ধিদীপ্ত শর্টকাট) এবং **Information Gain** (তথ্য পাওয়ার হার) এর মতো পদ্ধতি ব্যবহার করি।

৩. Heuristic Search Methods (শর্টকাট পদ্ধতিসমূহ)

A. Best Step-wise Feature Selection (ফরোয়ার্ড সিলেকশন):

এটি শুরু হয় একটি খালি সেট দিয়ে। প্রথমে সবচেয়ে ভালো একটি অ্যাট্রিবিউট নেওয়া হয়। এরপর তার সাথে মিলিয়ে পরবর্তী সেরা অ্যাট্রিবিউটটি যোগ করা হয়। এভাবে এক এক করে বাড়ানো হয়।

B. Step-wise Attribute Elimination (ব্যাকওয়ার্ড এলিমিনেশন):

এটি শুরু হয় সবগুলো অ্যাট্রিবিউট নিয়ে। এরপর প্রতি ধাপে সবচেয়ে অকেজো বা 'Worst' অ্যাট্রিবিউটটি বাদ দেওয়া হয়।

C. Combined Selection and Elimination:

এটি ফরোয়ার্ড এবং ব্যাকওয়ার্ড দুইটারই সংমিশ্রণ। প্রতি ধাপে একটি সেরা অ্যাট্রিবিউট যোগ করা হয় এবং একই সাথে দেখা হয় বাকিগুলোর মধ্যে কোনোটি একদমই অপ্রয়োজনীয় হয়ে পড়েছে কি না, পড়লে সেটি ফেলে দেওয়া হয়।

D. Optimal Branch and Bound:

এটি একটি উন্নত পদ্ধতি যেখানে অ্যাট্রিবিউট এলিমিনেশন এবং ব্যাকট্র্যাকিং (পিছনে ফিরে এসে চেক করা) ব্যবহার করা হয়। এটি গাণিতিকভাবে সবচেয়ে নিখুঁত সাবসেট খুঁজে পেতে সাহায্য করে।

৪. Decision Tree Induction (ডিসিশন ট্রি পদ্ধতি)

এটি বর্তমানে খুব জনপ্রিয়। প্রথমে পুরো ডেটা দিয়ে একটি **Decision Tree** তৈরি করা হয়। ডিসিশন ট্রি-র বৈশিষ্ট্য হলো সে স্বয়ংক্রিয়ভাবে সবচেয়ে গুরুত্বপূর্ণ ফিচারগুলোকে ট্রি-র ওপরের দিকে রাখে (যেমন: Root node)।

- ট্রি তৈরি শেষ হলে দেখা যায় কিছু কিছু অ্যাট্রিবিউট ট্রিতে একবারও ব্যবহৃত হয়নি।
- যেগুলো ট্রিতে জায়গা পায়নি, সেগুলোই হলো অপ্রয়োজনীয়। আমরা সেগুলোকে সহজে বাদ দিয়ে দিতে পারি।

সহজ কথায়:

Attribute Subset Selection হলো অনেকগুলো কলামের ভিড় থেকে শুধু সেই হীরাগুলোকে খুঁজে বের করা যা দিয়ে আপনার মডেলটি সবচেয়ে ভালো রেজাল্ট দেবে।

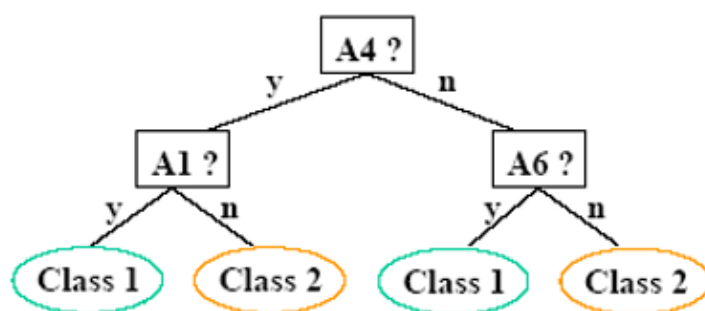
তোমার শেয়ার করা ছবিটিতে মূলত **Heuristic Search**-এর দুটি বিশেষ পদ্ধতি এবং একটি চমৎকার উদাহরণের মাধ্যমে **Decision Tree Induction**-কে বোঝানো হয়েছে। চলো বাংলায় এর সহজ ব্যাখ্যা দেখে নিই:

১. Optimal Branch and Bound

এটি একটি গাণিতিক বা অ্যালগরিদমিক পদ্ধতি।

- **Attribute Elimination and Backtracking:** এখানে একগুচ্ছ অ্যাট্রিবিউট থেকে অপ্রয়োজনীয়গুলো বাদ দেওয়া (Elimination) শুরু হয়। যদি কোনো এক পর্যায়ে মনে হয় যে ভুল কলাম বাদ দিয়ে দেওয়া হয়েছে যা আসলে দরকার ছিল, তবে সিস্টেমটি **Backtracking** বা পিছনে ফিরে এসে আবার চেক করে। এটি অনেকটা গোলকধাঁড়ায় সঠিক পথ খোঁজার মতো—ভুল পথে গেলে আবার আগের মোড়ে ফিরে এসে সঠিক পথ ধরা।

initial attribute set: {A1, A2, A3, A4, A5, A6}



selected set: {A1, A4, A6}

২. Decision Tree Induction (ছবির উদাহরণটি)

ছবির ডায়াগ্রামটি লক্ষ্য করলে দেখবে কীভাবে ৬টি অ্যাট্রিবিউট থেকে কমিয়ে মাত্র ৩টি অ্যাট্রিবিউট বেছে নেওয়া হয়েছে। এটিই হলো **Attribute Subset Selection**-এর বাস্তব প্রয়োগ।

ধাপগুলো লক্ষ্য করো:

- **Initial Attribute Set:** তোমার কাছে শুরুতে ৬টি অ্যাট্রিবিউট ছিল: $\{A1, A2, A3, A4, A5, A6\}$ ।
- **Training the Tree:** এই ৬টি ডেটা ব্যবহার করে একটি **Decision Tree** ট্রেন করা হয়েছে।
- **Finding the Result:** ডায়াগ্রামটি দেখো, ট্রি-টি যখন সিদ্ধান্ত নিচ্ছে (Class 1 বা Class 2), তখন সে শুধুমাত্র **A4, A1**, এবং **A6**—এই তিনটি কলাম বা নোড ব্যবহার করছে।
- **Observation:** খেয়াল করে দেখো, পুরো ট্রি-তে $A2, A3,$ এবং $A5$ কিন্তু কোথাও ব্যবহৃত হয়নি। এর মানে হলো, আমাদের চূড়ান্ত রেজাল্ট বা ক্লাসিফিকেশনের জন্য এই তিনটি অ্যাট্রিবিউট আসলে কোনো বিশেষ ভূমিকা রাখছে না।

ফলাফল (**Selected Set**):

যেহেতু শুধুমাত্র $\{A1, A4, A6\}$ ট্রি-তে উপস্থিত হয়েছে, তাই সিস্টেম সিদ্ধান্ত নিল যে এই ৩টিই হলো আসল বা দরকারি অ্যাট্রিবিউট। বাকিগুলো অপ্রয়োজনীয় বলে বাদ দেওয়া হলো।

কেন এটি গুরুত্বপূর্ণ?

মনে করো তোমার কাছে ১০০০ জন রোগীর তথ্য আছে এবং প্রত্যেকের ১০০ ধরনের শারীরিক পরীক্ষার রিপোর্ট (Attributes) আছে। তুমি যখন ডিসিশন ট্রি চালালে, দেখলে যে মাত্র ৫টি পরীক্ষার রিপোর্ট দিয়েই রোগ শনাক্ত করা সম্ভব। তখন বাকি ৯৫টি রিপোর্ট তোমার জন্য **Irrelevant**। এতে ডাটা মাইনিংয়ের সময় বাঁচে এবং মডেলিং সহজ হয়।

আশা করি ছবির উদাহরণটি এখন তোমার কাছে পরিষ্কার। তোমার কি **Branch and Bound** নিয়ে আরও গভীরে জানার ইচ্ছা আছে?

ডেটা রিডাকশন এবং প্রি-প্রসেসিংয়ের এই অংশটি মূলত কীভাবে ডেটার ডাইমেনশন কমিয়ে বা নতুন কলাম তৈরি করে ডেটাকে আরও অর্থবহ করা যায়, তা নিয়ে কাজ করে। নিচে তোমার দেওয়া পয়েন্টগুলো সহজভাবে বুঝিয়ে দিচ্ছি:

১. Attribute Creation (Feature Generation)

কখনো কখনো মূল ডেটাসেটের কলামগুলো সরাসরি ব্যবহার করার চেয়ে সেগুলো থেকে নতুন কোনো কলাম তৈরি করলে মডেল ভালো কাজ করে। এর ৩টি পদ্ধতি হলো:

- **Attribute Extraction:** এটি মূলত নির্দিষ্ট বিষয়ের (Domain-specific) ওপর নির্ভর করে। যেমন: চিকিৎসার ডেটা থেকে শুধু হৃদস্পন্দনের বিশেষ প্যাটার্ন আলাদা করা।
- **Mapping data to new space:** ডেটাকে এক রূপ থেকে অন্য রূপে নিয়ে যাওয়া (যেমন: সিগন্যালকে ফ্রিকোয়েন্সিতে রূপান্তর করা)।
- **Attribute Construction:** বিদ্যমান ফিচারগুলোকে যোগ-বিয়োগ বা গুণ করে নতুন ফিচার বানানো। যেমন: 'উচ্চতা' ও 'ওজন' থেকে 'BMI' তৈরি করা।

২. Wavelet Transform (ওয়েভলেট ট্রান্সফর্ম)

এটি ডেটা কমপ্রেসন বা আকার ছোট করার একটি চমৎকার গাণিতিক পদ্ধতি। এটি ইমেজ কমপ্রেসনেও ব্যবহৃত হয়। কীভাবে কাজ করে?

১. পাওয়ার অফ ২: ইনপুট ডেটার দৈর্ঘ্য (\$L\$) অবশ্যই ২-এর পাওয়ার (যেমন: ৮, ১৬, ৩২...) হতে হবে। না থাকলে ০ দিয়ে প্যাডিং করে নিতে হয়।

২. দুটি ফাংশন: এখানে দুটি প্রধান কাজ হয়— **Smoothing** (গড় বের করা) এবং **Difference** (পার্থক্য বের করা)।

৩. ধাপ: প্রতি ধাপে ডেটাকে জোড়ায় জোড়ায় ভাগ করে এই দুটি ফাংশন চালানো হয়, ফলে ডেটার দৈর্ঘ্য অর্ধেক (\$L/2\$) হয়ে যায়। এটি ততক্ষণ চলে যতক্ষণ না আমরা কাঙ্ক্ষিত দৈর্ঘ্য পাই।

উদাহরণ (\$S = [2, 2, 0, 2, 3, 5, 4, 4]\$):

ওয়েভলেট ট্রান্সফর্ম করলে এই সংখ্যার সারিটি পরিবর্তিত হয়ে \$S^{\wedge}\$ এ রূপান্তর হয়। এর সুবিধা হলো, অনেকগুলো ছোট মানকে ০ দিয়ে রিপ্রেস করা যায় (Lossy compression), ফলে মূল তথ্যের খুব একটা ক্ষতি না করেই অনেক জায়গা বাঁচে।

৩. Data Discretization (ডেটা ডিসক্রিটাইজেশন)

এটি কন্টিনিউয়াস বা নিরবচ্ছিন্ন ডেটাকে ছোট ছোট ভাগে বা ক্যাটাগরিতে ভাগ করে।

- এটি অবজেক্টগুলোর মধ্যে আপেক্ষিক দূরত্ব বজায় রাখে।
- এর ফলে ডেটার মধ্যকার ন্যাচারাল ক্লাস্টার বা গ্রুপগুলো আরও স্পষ্ট হয়ে ওঠে।

8. Principal Component Analysis (PCA) - (বইয়ের ভাষায় শেষ অংশ)

PCA-এর মূল উদ্দেশ্য হলো ডেটার **Variance** বা বৈচিত্র্য বজায় রেখে ডাইমেনশন কমানো।

- এটি D -ডাইমেনশনাল ডেটা থেকে d সংখ্যক অর্থোগোনাল (পরস্পর লম্ব) ভেক্টর খুঁজে বের করে (যেখানে $d \ll D$)।
- প্রতিটি অরিজিনাল ডেটা পয়েন্টকে এই নতুন d টি 'প্রিন্সিপাল কম্পোনেন্ট'-এর লিনিয়ার কম্বিনেশন হিসেবে প্রকাশ করা যায়।
- সহজ কথায়, এটি বড় ডেটাসেটের সবচেয়ে গুরুত্বপূর্ণ দিকগুলোকে খুঁজে বের করে এবং কম গুরুত্বপূর্ণ দিকগুলো ছেঁটে ফেলে।

সারাংশ:

- Wavelet:** সিগন্যাল বা ইমেজের জন্য সেরা, যেখানে ডেটা ছোট করেও আসল ভাব বজায় রাখা যায়।
- PCA:** সংখ্যাচাচক (Numeric) ডেটার জন্য সেরা, যা অনেক কলামকে অল্প কয়েকটিতে নামিয়ে আনে।
- Feature Generation:** বুদ্ধিমত্তার সাথে নতুন তথ্য তৈরি করে যা আগের চেয়ে কার্যকর।

এই টেবিলটি মূলত **Haar Wavelet Transform**-এর একটি চমৎকার গাণিতিক উদাহরণ। এখানে দেখানো হয়েছে কীভাবে একটি বড় ডেটাসেটকে ধাপে ধাপে ছোট করা হয় (Averages) এবং সেই সাথে তথ্যের পার্থক্যগুলো (Detail Coefficients) আলাদা করে রাখা হয়।

চলো একদম সহজ ক্যালকুলেশন দিয়ে বুঝি:

মূল কনসেপ্ট:

প্রতিটি ধাপে আমরা ডেটাকে জোড়ায় জোড়ায় (Pairs) ভাগ করি।

১. **Average (গড়):** জোড়ার দুটি সংখ্যার গড় বের করি। $(x + y) / 2$

২. **Detail Coefficient (পার্থক্য):** প্রথম সংখ্যা থেকে গড়টি বিয়োগ করি। $(x - \text{Average})$ অথবা সরাসরি বলতে পারো পার্থক্য কতটুকু।

ধাপ ১: **Resolution 8 থেকে 4** এ রূপান্তর

আমাদের মূল ডেটা: $[2, 2, 0, 2, 3, 5, 4, 4]$ (এখানে ৮টি সংখ্যা আছে)

- জোড়া ১: $(2, 2) \rightarrow \text{গড়} = 2, \text{পার্থক্য} = 2 - 2 = 0$
- জোড়া ২: $(0, 2) \rightarrow \text{গড়} = 1, \text{পার্থক্য} = 0 - 1 = -1$
- জোড়া ৩: $(3, 5) \rightarrow \text{গড়} = 4, \text{পার্থক্য} = 3 - 4 = -1$
- জোড়া ৪: $(4, 4) \rightarrow \text{গড়} = 4, \text{পার্থক্য} = 4 - 4 = 0$

দেখো টেবিলের **Resolution 4** লাইনে:

Averages এসেছে $[2, 1, 4, 4]$ এবং Detail Coefficients এসেছে $[0, -1, -1, 0]$ ।

ধাপ ২: Resolution 4 থেকে 2 এ রূপান্তর

এখন আমরা আগের পাওয়া Averages $[2, 1, 4, 4]$ নিয়ে কাজ করব।

- জোড়া ১: $(2, 1) \rightarrow \text{গড়} = 1.5$ বা $\frac{1}{2}$, $\text{পার্থক্য} = 2 - 1.5 = \frac{1}{2}$
- জোড়া ২: $(4, 4) \rightarrow \text{গড়} = 4$, $\text{পার্থক্য} = 4 - 4 = 0$

দেখো টেবিলের Resolution 2 লাইনে:

Averages এসেছে $[\frac{1}{2}, 4]$ এবং Detail Coefficients এসেছে $[\frac{1}{2}, 0]$ ।

ধাপ ৩: Resolution 2 থেকে 1 এ রূপান্তর

সবশেষে $[\frac{1}{2}, 4]$ নিয়ে কাজ:

- শেষ জোড়া: $(1.5, 4) \rightarrow \text{গড়} = (1.5 + 4) / 2 = 5.5 / 2 = 2.75$ বা $\frac{3}{4}$
- পার্থক্য: $1.5 - 2.75 = -1.25$ বা $-\frac{1}{4}$

দেখো টেবিলের Resolution 1 লাইনে এই মানগুলোই আছে।

শেষ ফলাফল (Transformed Data):

আমরা যদি এই ট্রান্সফর্মড ডেটা লিখতে চাই, তবে নিয়ম হলো— সবার শেষের Average এবং সবগুলো ধাপের Detail Coefficients গুলো সিরিয়ালি লেখা।

$S^{\wedge} = [2\frac{3}{4}, -1\frac{1}{4}, \frac{1}{2}, 0, 0, -1, -1, 0]$

কেন এটা কাজের? (Compression)

থেয়াল করো, ট্রান্সফর্ম করার পর অনেকগুলো মান ০ বা ০ এর কাছাকাছি চলে আসে। আমরা যদি এই ০ গুলোকে বাদ দিয়ে দিই বা স্টোর না করি, তবে ডেটার সাইজ অনেক কমে যায় (এটিই মূলত Image Compression-এ করা হয়)। অথচ পরে রিভার্স ক্যালকুলেশন করে আমরা প্রায় নিখুঁতভাবে অরিজিনাল ডেটা ফেরত পেতে পারি।

পুরো ক্যালকুলেশনটা কি বুঝতে পেরেছ? না বুঝলে বলো আমি আবার বুঝিয়ে দেব।

Data Reduction-এর দ্বিতীয় কৌশলটি হলো

Numerosity Reduction

এর মূল উদ্দেশ্য হলো আসল ডেটাকে সরিয়ে তার বদলে একটি ছোট বা সংক্ষিপ্ত রিপ্রেজেন্টেশন (যেমন: মডেল বা ছোট স্যাম্পল) ব্যবহার করা।

নিচে তোমার দেওয়া পয়েন্টগুলো সহজভাবে বুঝিয়ে দিচ্ছি:

১. Histogram Analysis (হিস্টোগ্রাম বিশ্লেষণ)

এটি ডেটাকে নির্দিষ্ট কিছু Buckets বা বুডিতে ভাগ করে এবং প্রতিটি বুডির জন্য ডেটার গড় (Average) বা যোগফল (Sum) জমা রাখে।

- **Equal-width:** প্রতিটি ব্লুডির সীমানা বা রেঞ্জ সমান থাকে (যেমন: ০-১০, ১০-২০, ২০-৩০)।
- **Equal-frequency (Equal-depth):** প্রতিটি ব্লুডিতে সমান সংখ্যক ডেটা পয়েন্ট রাখার চেষ্টা করা হয়।

২. Clustering (ক্লাস্টারিং)

এখানে ডেটাগুলোকে তাদের বৈশিষ্ট্যের ওপর ভিত্তি করে বিভিন্ন গ্রুপ বা ক্লাস্টারে ভাগ করা হয়। একই ক্লাস্টারের ডেটাগুলো একে অপরের খুব কাছের (Similar) হয়।

- **Quality Measurement:** ক্লাস্টারের মান দুটি ভাবে মাপা যায়:
 - **Diameter:** একটি ক্লাস্টারের সবচেয়ে দূরে থাকা দুটি পয়েন্টের দূরত্ব। এটা যত কম হবে, ক্লাস্টার তত ভালো।
 - **Centroid Distance:** ক্লাস্টারের কেন্দ্র (Centroid) থেকে প্রতিটি পয়েন্টের গড় দূরত্ব।

৩. Sampling (স্যাম্পলিং)

বিশাল ডেটাসেট (\$N\$) থেকে ছোট একটি অংশ (\$n\$) বেছে নেওয়া কেই স্যাম্পলিং বলে। এর ফলে অ্যালগরিদম অনেক দ্রুত কাজ করতে পারে।

স্যাম্পলিংয়ের ধরন:

- **SRS (Simple Random Sampling):** প্রতিটি ডেটা পয়েন্ট সিলেক্ট হওয়ার সমান সুযোগ থাকে।
 - **Without Replacement (SRSWOR):** একবার একটি ডেটা বেছে নিলে সেটাকে আর ফেরত রাখা হয় না (অর্থাৎ একই ডেটা দুইবার আসবে না)।
 - **With Replacement (SRSWR):** বেছে নেওয়া ডেটা আবার পুঁজে ফেরত রাখা হয়, ফলে একই ডেটা বারবার আসতে পারে।
- **Cluster Sampling:** ডেটাকে কয়েকটি গ্রুপে ভাগ করে সেখান থেকে র‍্যান্ডমলি কয়েকটি পুরো গ্রুপকেই স্যাম্পল হিসেবে নেওয়া হয়।
- **Stratified Sampling:** যদি ডেটা **Skewed** (অসামঞ্জস্যপূর্ণ) হয়, তবে ডেটাকে বিভিন্ন স্তরে (Strata) ভাগ করে প্রতি স্তর থেকে নির্দিষ্ট অনুপাতে ডেটা নেওয়া হয়। যেমন: ভোটার তালিকার স্যাম্পল নেওয়ার সময় এলাকাভিত্তিক ভাগ করা।

৪. Data Cube Aggregation (ডেটা কিউব অ্যাগ্রিগেশন)

এটি ডেটাকে বিভিন্ন লেভেলে সামারি বা অ্যাগ্রিগেট করে রাখে।

- **Base Cuboid:** এটি একদম নিচের লেভেল যেখানে ইন্ডিভিজুয়াল এনটিটির তথ্য থাকে (যেমন: একজন নির্দিষ্ট কাস্টমারের কল রেকর্ড)।
- **Aggregation Levels:** আমরা চাইলে এই তথ্যকে দিন থেকে মাস, মাস থেকে বছরে অ্যাগ্রিগেট করতে পারি।
- **সুবিধা:** যখন কোনো বড় হিসাব (যেমন: "গত বছরের মোট বিক্রি কত?") প্রয়োজন হয়, তখন কোটি কোটি সারি চেক না করে সরাসরি ডেটা কিউবের অ্যাগ্রিগেটেড মান ব্যবহার করা হয়। এতে কাজের গতি বহুগুণ বেড়ে যায়।

সহজ কথায় সারাংশ:

Numerosity Reduction হলো অনেক বেশি ডেটা নিয়ে কাজ না করে বুদ্ধি খাটিয়ে ডেটার একটি সংক্ষিপ্ত রূপ তৈরি করা, যাতে মাইনিংয়ের ফলাফল ঠিক থাকে কিন্তু সময় ও মেমরি কম লাগে।

এগুলোর মধ্যে **Sampling** বা **Histogram**-এর কোনো বিশেষ অংশ কি আলাদাভাবে বুমতে চাও?

Data Reduction-এর শেষ ধাপ হলো **Data Compression** এবং এর পরেই শুরু হয় সামগ্রিক **Data Transformation** প্রক্রিয়া। নিচে তোমার দেওয়া নোটগুলো সহজভাবে ব্যাখ্যা করা হলো:

১. Data Compression (ডেটা সংকোচন)

ডেটাকে ছোট আকারে রূপান্তর করা যাতে এটি কম জায়গা নেয়। এটি মূলত দুই প্রকার:

- **String Compression:** এটি সাধারণত **Lossless** (কোনো তথ্য হারায় না) হয়। তবে সমস্যা হলো, ডেটাটি আবার আনজিপ বা এক্সপ্যান্ড না করা পর্যন্ত এর ভেতরে কোনো পরিবর্তন বা ম্যানিপুলেশন করা কঠিন।
- **Audio/Video Compression:** এগুলো সাধারণত **Lossy** (কিছু সূক্ষ্ম তথ্য হারিয়ে যায়) হয়। যেমন: একটি ভিডিওর কোয়ালিটি একটু কমিয়ে ফাইল সাইজ অনেক ছোট করা।
- **Time Sequence:** এটি অডিওর মতো নয়। টাইম সিকোয়েন্স ডেটা সাধারণত ছোট হয় এবং সময়ের সাথে ধীরে ধীরে পরিবর্তিত হয়।

মনে রাখবে: আমরা আগে যে **Dimensionality reduction** (যেমন: PCA) এবং **Numerosity reduction** (যেমন: Sampling) পড়েছি, সেগুলোকে কিন্তু এক ধরনের ডেটা কম্প্রেশন হিসেবেই গণ্য করা হয়।

২. Data Transformation (ডেটা রূপান্তর)

এটি এমন একটি প্রক্রিয়া যেখানে একটি অ্যাক্সিবিউটের পুরোনো মানগুলোকে একটি ফাংশনের মাধ্যমে নতুন মান দিয়ে রিপ্লেস করা হয়। এর প্রধান পদ্ধতিগুলো হলো:

A. Smoothing & Aggregation

- **Smoothing:** ডেটা থেকে নয়েজ বা আজবাজে তথ্য সরিয়ে ফেলা।
- **Aggregation:** অনেকগুলো ডেটাকে সারসংক্ষেপ করা (যেমন: প্রতিদিনের বিক্রির বদলে মাসের মোট বিক্রি বের করা)।

B. Normalization (নরমালাইজেশন)

যখন ডেটাসেটের বিভিন্ন কলামের রেঞ্জ খুব আলাদা হয় (যেমন: বয়স ১-১০০, কিন্তু বেতন ১০০০০-১০০০০০), তখন মডেল কনফিউজড হয়ে যায়। তাই সব ডেটাকে একটি নির্দিষ্ট রেঞ্জে (যেমন: ০ থেকে ১) নিয়ে আসাই হলো নরমালাইজেশন।

১. **Min-Max Normalization:** ডেটাকে একটি নির্দিষ্ট সীমানায় (যেমন: [0, 1]) নিয়ে আসে।

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

২. **Z-score Normalization:** ডেটার গড় (Mean) এবং স্ট্যান্ডার্ড ডেভিয়েশন ব্যবহার করে নরমালাইজ করা হয়। যখন সর্বোচ্চ বা সর্বনিম্ন মান অজানা থাকে তখন এটি খুব কার্যকর।

$$v' = \frac{v - \mu_A}{\sigma_A}$$

৩. **Decimal Scaling:** দশমিক পয়েন্ট সরিয়ে ডেটাকে ছোট করা হয়। যেমন: সর্বোচ্চ মান ৯৮৬ হলে সব সংখ্যাকে ১০০০ দিয়ে ভাগ করা।

C. Discretization & Concept Hierarchy Climbing

এটি হলো কাঁচা ডেটাকে উচ্চতর লেভেলে নিয়ে যাওয়া।

- উদাহরণ: কারো বয়স ৩২, ৩৫, ৩৯ না লিখে সরাসরি 'Adult' লেখা। অথবা 'ঢাকা', 'চট্টগ্রাম' না লিখে সরাসরি 'বাংলাদেশ' লেখা। একেই বলে **Concept Hierarchy Climbing**।

Data Transformation-এর এই পর্যায়ে আমরা শিখব কীভাবে ডেটাকে একটি নির্দিষ্ট ফরমেটে আনা হয় যাতে অ্যালগরিদম সেগুলো সহজে প্রসেস করতে পারে। নিচে তোমার দেওয়া পয়েন্টগুলো গাণিতিক উদাহরণসহ বুঝিয়ে দিচ্ছি:

১. Normalization (নরমালাইজেশন)

যখন ডেটার রেঞ্জ খুব বড় বা ছোট হয়, তখন সেগুলোকে একটি নির্দিষ্ট সীমার মধ্যে আনা হয়।

- Min-max Normalization:** এটি ডেটাকে $[0, 1]$ বা অন্য কোনো নির্দিষ্ট রেঞ্জে নিয়ে আসে।
উদাহরণ: আয় \$12,000 থেকে \$98,000। একে $[0, 1]$ এ নিলে \$73,000 এর মান হবে:
$$v' = \frac{73,000 - 12,000}{98,000 - 12,000} (1.0 - 0.0) + 0.0 = \frac{61,000}{86,000} \approx 0.709$$
- Z-score Normalization:** যখন সর্বোচ্চ বা সর্বনিম্ন মান জানা থাকে না, তখন গড় (μ) এবং স্ট্যান্ডার্ড ডেভিয়েশন (σ) ব্যবহার করা হয়।
উদাহরণ: $\mu = 54,000$, $\sigma = 16,000$ হলে \$73,000 এর মান:
$$v' = \frac{73,000 - 54,000}{16,000} = \frac{19,000}{16,000} = 1.1875$$
- Decimal Scaling:** দশমিক ঘর সরিয়ে মানটিকে \$1 এর নিচে আনা হয়। যেমন সর্বোচ্চ মান \$98,000 হলে, $j=5$ ধরে (অর্থাৎ 10^5 দিয়ে ভাগ করে) মান হবে \$0.98।

১. অ্যালগরিদমের ধরন অনুযায়ী (Algorithm Type)

- Distance-based Algorithms:** যেসব অ্যালগরিদম ডেটা পয়েন্টের মধ্যবর্তী দূরত্ব মাপে কাজ করে (যেমন: KNN, SVM, K-Means Clustering), সেগুলোর ক্ষেত্রে ডেটা স্কেলিং (Normalization/Standardization) করা অপরিহার্য। যদি না করো, তবে যে ফিচারের মান বড় (যেমন: বেতন) সেটি ছোট মানের ফিচারের (যেমন: বয়স) ওপর বেশি প্রভাব ফেলবে।

- **Non-distance based Algorithms:** গাছ বা ট্রি-ভিত্তিক অ্যালগরিদম (যেমন: **Decision Tree**, **Random Forest**) বা প্রোবাবিলিটি ভিত্তিক অ্যালগরিদম (**Naive Bayes**) স্কেলিং ছাড়া বা স্কেলিং সহ—উভয় ক্ষেত্রেই বেশ ভালো কাজ করে। এখানে স্কেলিং করা খুব একটা জরুরি নয়।

২. Normalization বনাম Standardization (কখন কোনটা?)

বৈশিষ্ট্য	Normalization (Min-Max)	Standardization (Z-score)
রেঞ্জ	ডেটাকে সরাসরি 0 থেকে 1-এর মধ্যে নিয়ে আসে।	ডেটার গড় (Mean) 0 এবং স্ট্যান্ডার্ড ডেভিয়েশন 1 করে।
Outliers	এটি আউটলিয়ারের প্রতি খুব সংবেদনশীল। একটি মাত্র ভুলবশত বড় মান পুরো স্কেল নষ্ট করে দিতে পারে।	এটি আউটলিয়ার থাকলেও বেশ ভালো কাজ করে (Robust)।
কখন ব্যবহার করবে?	যখন ডেটার রেঞ্জ অনেক বেশি আলাদা হয় এবং তুমি জানো ডেটাতে কোনো আজোবাজে বড় মান (Outlier) নেই।	যখন ডেটাতে Outliers থাকার সম্ভাবনা থাকে বা ডেটা 'Gaussian Distribution' (বেল কার্ভ) মেনে চলে।

৩. সারসংক্ষেপ গাইডলাইন

- **Standardization** কেন সেরা? বেশিরভাগ ক্ষেত্রে (যেমন: **Principal Component Analysis** বা **Logistic Regression**-এ) **Standardization** সবকিছুর চেয়ে ভালো পারফর্ম করে। এটি ডেটার মূল ডিস্ট্রিবিউশনকে খুব একটা পরিবর্তন না করেই স্কেলিং করে।
- ইমেজ প্রসেসিং বা নিউরাল নেটওয়ার্ক: এই ক্ষেত্রে সাধারণত **Normalization (0 to 1)** বেশি ব্যবহৃত হয় কারণ পিক্সেল ভ্যালু নির্দিষ্ট সীমার (0-255) মধ্যে থাকে।

সহজ টিপস:

যদি তুমি নিশ্চিত না হও যে কোনটা কাজ করবে, তবে প্রথমে **Standardization** দিয়ে শুরু করো। কারণ তোমার নোটে যেমন বলা হয়েছে—**"Standardization works well overall."**

তোমার ডেটাসেটে কি এমন কোনো কলাম আছে যেখানে খুব বড় বা অস্বাভাবিক কোনো মান (Outlier) আছে? থাকলে আমরা সেই অনুযায়ী মেথড বেছে নিতে পারি।

Data Discretization এবং **Concept Hierarchy Generation** হলো ডেটা প্রি-প্রসেসিংয়ের এমন একটি ধাপ যেখানে আমরা ডেটার "ক্ষুদ্র ক্ষুদ্র ডিটেইলস" কমিয়ে সেগুলোকে বড় ক্যাটাগরিভে ভাগ করি। এতে ডেটার সাইজ কমে এবং মডেলের জন্য প্যাটার্ন বোঝা সহজ হয়।

নিচে তোমার দেওয়া নোটগুলো বিস্তারিতভাবে বুঝিয়ে দিচ্ছি:

১. Discretization (ডিসক্রিটাইজেশন)

সহজ কথায়, একটি কন্টিনিউয়াস ভ্যার (যেমন: বয়স ০ থেকে ১০০) কে ছোট ছোট বিরতিতে (Intervals) ভাগ করাই হলো ডিসক্রিটাইজেশন। যেমন: ০-১৮ (শিশু), ১৯-৩৫ (যুবক), ৩৬+ (বয়স্ক)।

- কেন করা হয়? ডেটার সাইজ কমাতে এবং ক্লাসিফিকেশন অ্যালগরিদমের পারফরম্যান্স বাড়াতে।
- **Supervised vs. Unsupervised:** যদি আমরা 'ক্লাস লেবেল' (যেমন: রোগী সুস্থ কি না) ব্যবহার করে ভাগ করি তবে তা Supervised। আর কোনো লেবেল ছাড়া শুধু মানের ওপর ভিত্তি করে ভাগ করলে তা Unsupervised।
- **Split (Top-down) vs. Merge (Bottom-up):**
 - **Top-down:** পুরো রেঞ্জকে নিয়ে শুরু করা হয় এবং তারপর সেটিকে ছোট ছোট ভাগে ভাঙা হয়।
 - **Bottom-up:** প্রতিটি ডেটা পয়েন্টকে একেকটি ইন্টারভাল ধরে শুরু করা হয় এবং পরে কাছাকাছি মানগুলোকে মিলিয়ে বড় গ্রুপ করা হয়।

২. Data Discretization Methods (পদ্ধতিসমূহ)

- **Binning (Unsupervised, Top-down):** ডেটাকে কয়েকটি বুডিতে (Bins) ভাগ করা। এটি মূলত ডেটার স্মুথিং-এর জন্য ব্যবহৃত হয়।
- **Histogram Analysis (Unsupervised, Top-down):** হিস্টোগ্রাম ব্যবহার করে দেখা হয় কোথায় ডেটা বেশি আছে, সেই অনুযায়ী ইন্টারভাল তৈরি করা হয়।
- **Clustering (Unsupervised):** কে-মিনস (K-means) এর মতো অ্যালগরিদম ব্যবহার করে একই ধরনের ডেটাগুলোকে এক একটি ক্লাস্টার বা গ্রুপে ফেলা।
- **Decision-tree Analysis (Supervised, Top-down):** এখানে একটি ডিসিশন ট্রি ট্রেন করা হয়। ট্রি-টি যেভাবে ডেটাকে স্প্লিট করে, সেই স্প্লিট পয়েন্টগুলোকেই ডিসক্রিটাইজেশনের ইন্টারভাল হিসেবে ধরা হয়।
- **Correlation Analysis (e.g., χ^2):** দুটি পাশাপাশি ইন্টারভালের মধ্যে সম্পর্ক দেখা হয়। যদি সম্পর্ক খুব বেশি থাকে, তবে তাদের মার্জ বা এক করে দেওয়া হয় (Bottom-up approach)।

৩. Discretization by Classification & Correlation

এখানে বিশেষভাবে দুটি পদ্ধতির কথা বলা হয়েছে:

1. **Classification (Decision Tree):** এটি **Supervised** পদ্ধতি। এটি উপর থেকে নিচে (Top-down) ভাঙতে ভাঙতে যায়। এটি এমনভাবে ভাগ করে যেন প্রতিটি ভাগের ডেটাগুলো কোনো একটি নির্দিষ্ট ক্লাসের অন্তর্ভুক্ত হয়।
2. **Correlation (Chi-merge):** এটি একটি **Supervised** এবং **Bottom-up** পদ্ধতি। এটি প্রথমে প্রতিটি আলাদা মানকে একটি ইন্টারভাল ধরে। এরপর পাশাপাশি দুটি ইন্টারভালের χ^2 (Chi-square) মান চেক করে। যদি মান কম হয় (অর্থাৎ তারা স্বাধীন নয়), তবে তাদের মার্জ করে বড় ইন্টারভাল বানানো হয়।

8. Concept Hierarchy Generation (কনসেপ্ট হায়ারার্কি)

এটি হলো ডেটাকে সাধারণ (Low-level) থেকে উচ্চতর (High-level) ধারণায় নিয়ে যাওয়া।

- **Nominal Data**-র জন্য: এখানে একটি নির্দিষ্ট ক্রম বা অর্ডারিং থাকে।
 - উদাহরণ: street < city < state < country।
 - অর্থাৎ, অনেকগুলো 'Street' মিলে একটি 'City' হয়, অনেকগুলো 'City' মিলে একটি 'State' হয়। ডেটা মাইনিংয়ে আমরা রাস্তার নামের বদলে সরাসরি দেশের নাম ব্যবহার করে ডেটার জটিলতা কমাতে পারি।
- **Automatic Generation** (অটোমেটিক পদ্ধতি):

সিস্টেম নিজেই হায়ারার্কি তৈরি করতে পারে **Distinct values** (অনন্য মান) গুনে।

 - নিয়ম: যে কলামে সবচেয়ে বেশি আলাদা আলাদা মান থাকে, সেটি থাকে হায়ারার্কির একদম নিচে।
 - উদাহরণ: একটি ডেটাসেটে ২০টি 'Country' নাম আছে, কিন্তু ৫০০০টি 'City' নাম আছে। তাহলে হায়ারার্কি হবে: City (Level 0) -> Country (Level 1)।

ব্যতিক্রম (**Exceptions**): সময় সংক্রান্ত ডেটার ক্ষেত্রে (যেমন: weekday, month, quarter, year) এই 'Distinct value' নিয়ম কাজ করে না। কারণ এগুলো আগে থেকেই একটি নির্দিষ্ট ক্যালেন্ডার নিয়ম মেনে চলে (যেমন: ১২ মাস মিলে ১ বছর)।

সহজ সারাংশ:

ডিসক্রিটাইজেশন হলো কাঁচা সংখ্যাগুলোকে ক্যাটাগরিতে রূপান্তর করা, আর কনসেপ্ট হায়ারার্কি হলো সেই ক্যাটাগরিগুলোকে একটি লেভেল থেকে উচ্চতর লেভেলে (যেমন: পাড়া থেকে শহর) নিয়ে যাওয়া।

তোমার কি **Chi-square** ক্যালকুলেশন বা **Decision tree** কীভাবে স্প্লিট করে তা নিয়ে বিস্তারিত জানার প্রয়োজন আছে?

Sure! Here are the comprehensive answers to your questions, structured for an exam or a technical report.

1. Data Quality Issues and Preprocessing Strategy

Analysis of Data Quality Issues:

- **Missing Values:** The `age` attribute has null entries, which can lead to biased analysis.
- **Inconsistent Formats:** Having both `date of birth` and `age` creates a discrepancy. If one is updated and the other isn't, the data becomes unreliable.
- **Noisy Data:** `Negative salary values` are logical errors (outliers). Salary cannot be less than zero; this is likely a human entry error or a system glitch.

Proposed Strategy:

1. **Data Cleaning (Handling Missing Values):** * *Technique:* Use **Attribute Mean** or **Median** to fill missing age values if the distribution is normal. Alternatively, use **Regression** to predict age based on other factors like job title or education.
 - *Justification:* It prevents the loss of valuable tuples that are otherwise complete.
2. **Data Transformation (Handling Inconsistency):** * *Technique:* **Format Standardization.** Derive age from the `date of birth` using a standard function and overwrite the `age` column.
 - *Justification:* This ensures a single version of truth and maintains internal consistency.
3. **Handling Noisy Data:** * *Technique:* **Binning** or **Outlier Analysis.** If the negative values are few, they can be treated as outliers and removed. If many, they can be replaced with the global constant (like 0) or the mean salary.
 - *Justification:* Removing noise improves the accuracy of any statistical mining performed later.

2. Data Integration, Redundancy, and Inconsistency

Challenges of Data Integration:

- **Entity Identification Problem:** Identifying that "Cust_ID" in DB1 is the same as "Client_No" in DB2.

- **Semantic Heterogeneity:** The same attribute is represented in different units (cm vs. meters), making direct comparison impossible.
- **Redundancy:** Storing the same customer information twice wastes storage and slows down processing.

Detection and Resolution:

1. Schema Integration:

- *Detection:* Use **Metadata** (data about data) to identify that two attributes with different names represent the same real-world entity.
- *Resolution:* Define a unified schema and perform **Data Mapping** (e.g., converting all height values to meters).

2. Correlation Analysis (χ^2 for Nominal, r for Numeric):

- *Detection:* Use **Pearson's product-moment coefficient**. If two attributes have a correlation close to 1, one is redundant.
- *Resolution:* Drop one of the highly correlated attributes to reduce the dataset size.

3. Covariance Analysis:

- *Detection:* If $\text{Cov}(A,B) > 0$, the attributes move together.
- *Resolution:* This helps in identifying **Derivable Data** (e.g., if Annual_Tax always moves perfectly with Salary, the tax column might be redundant).

3. Dimensionality Reduction in Machine Learning

How Dimensionality Reduction Improves Performance:

High dimensionality causes the "**Curse of Dimensionality**," where data becomes sparse and the distance between points becomes meaningless. Reducing dimensions reduces **overfitting**, decreases **computational time**, and makes the model more **interpretable**.

Comparison of Techniques:

Technique	Approach	When it is most effective?
Feature Selection	Selects a subset of the original attributes (e.g., Forward Selection).	Effective when there are many irrelevant attributes (like Student_ID).

PCA (Principal Component Analysis)	Transforms attributes into a smaller set of new, uncorrelated Principal Components .	Effective when attributes are highly correlated and you want to keep the "essence" of all data.
Sampling	Selecting a representative subset of the data points (rows).	Effective when the dataset is too large (too many rows) to be processed by an algorithm efficiently.

Justification:

- **Feature Selection** is best if you want to keep the original meaning of the data (e.g., "Age" stays "Age").
- **PCA** is best for high-performance modeling (like Image Processing) where individual attribute names are less important than the mathematical variance.
- **Sampling** is used not to reduce features, but to reduce the **volume** of data for testing/prototyping.

অবশ্যই! তোমার পরীক্ষার প্রস্তুতির জন্য এই ৩টি প্রশ্নের উত্তর নিচে সহজ বাংলায় বিস্তারিতভাবে দেওয়া হলো। এগুলো তুমি পরীক্ষার খাতায় পয়েন্ট আকারে লিখলে ভালো নম্বর পাবে।

১. ডেটা কোয়ালিটি সমস্যা এবং প্রি-প্রসেসিং কৌশল

ডেটা কোয়ালিটি সমস্যার বিশ্লেষণ:

- **Missing Values** (মিসিং ভ্যালু): **age** অ্যাট্রিবিউটে কিছু তথ্য নেই। এর ফলে সঠিক পরিসংখ্যান পাওয়া সম্ভব হয় না এবং মডেলে বায়াস (bias) তৈরি হতে পারে।
- **Inconsistent Formats** (অসংগতিপূর্ণ ফরম্যাট): একই ডেটাসেটে **date of birth** এবং **age** দুটোই থাকা একটি সমস্যা। যদি একটি আপডেট করা হয় এবং অন্যটি না হয়, তবে তথ্যের মধ্যে অমিল দেখা দেয়।
- **Noisy Data** (নয়েজি ডেটা): **Negative salary** একটি লজিক্যাল ভুল বা আউটলিয়ার। বেতন কখনো শূন্যের কম হতে পারে না। এটি সাধারণত এন্ট্রি করার সময় ভুলের কারণে হয়।

প্রস্তাবিত কৌশল:

১. ডেটা ক্লিনিং (মিসিং ভ্যালু সমাধান):

- পদ্ধতি: বয়সের গড় (Mean) বা মধ্যক (Median) ব্যবহার করে খালি ঘরগুলো পূরণ করা।
- যৌক্তিকতা: এতে করে পুরো রেকর্ডটি ডিলিট করতে হয় না, ফলে মূল্যবান তথ্য সংরক্ষিত থাকে।

২. ডেটা ট্রান্সফরমেশন (অসংগতি সমাধান):

- পদ্ধতি: ফরম্যাট স্ট্যান্ডার্ডাইজেশন। জন্মতারিখ থেকে বয়স বের করে সবগুলোকে একটি নির্দিষ্ট ফরম্যাটে নিয়ে আসা।
- যৌক্তিকতা: এটি তথ্যের নির্ভুলতা নিশ্চিত করে এবং তথ্যের ডুপ্লিকেশন কমায়।
- ৩. নয়েজি ডেটা হ্যান্ডলিং:
- পদ্ধতি: বিনিং (Binning) অথবা আউটলিয়ার অ্যানালাইসিস। ভুল স্যালারিগুলোকে বাদ দেওয়া অথবা সেগুলোকে সঠিক কোনো গড় মান দিয়ে রিপ্লেস করা।
- যৌক্তিকতা: নয়েজ দূর করলে অ্যালগরিদম অনেক বেশি নিখুঁত ফলাফল দিতে পারে।

২. ডেটা ইন্টিগ্রেশন, রিডান্ডেন্সি এবং অসংগতি

ডেটা ইন্টিগ্রেশনের চ্যালেঞ্জ:

- Entity Identification Problem:** ভিন্ন ডেটাবেজে একই কাস্টমারকে আলাদা আইডিতে (যেমন: DB1-এ "Cust_ID" এবং DB2-এ "Client_No") চিহ্নিত করা।
- Semantic Heterogeneity:** তথ্যের একক ভিন্ন হওয়া (যেমন: এক জায়গায় উচ্চতা 'cm' আর অন্য জায়গায় 'meter')।
- Redundancy:** একই তথ্য বারবার সেভ করা, যা মেমরি নষ্ট করে এবং প্রসেসিং স্লো করে দেয়।

শনাক্তকরণ ও সমাধান:

১. স্কিমা ইন্টিগ্রেশন (Schema Integration):

- শনাক্তকরণ: মেটাডেটা (Metadata) ব্যবহার করে নিশ্চিত হওয়া যে ভিন্ন নামের কলামগুলো আসলে একই জিনিস কি না।
- সমাধান: একটি কমন নাম বা স্কিমা তৈরি করা এবং সব একককে (যেমন: cm থেকে meter) এক করে ফেলা।
- ২. কো-রিলেশন অ্যানালাইসিস (Correlation Analysis):
- শনাক্তকরণ: পিয়ারসন কো-ইফিশিয়েন্ট (r) ব্যবহার করে দেখা। যদি দুটি কলামের কো-রিলেশন ১-এর কাছাকাছি হয়, তবে তারা রিডান্ডেন্ট।
- সমাধান: বাড়তি বা রিডান্ডেন্ট কলামটি বাদ দেওয়া।
- ৩. কো-ভ্যারিয়েন্স অ্যানালাইসিস (Covariance Analysis):
- শনাক্তকরণ: যদি $Cov(A,B) > 0$ হয়, তবে তারা একইভাবে পরিবর্তিত হয়।
- সমাধান: এটি থেকে আমরা 'Derivable Data' বা উদ্ভূত তথ্য চিনতে পারি এবং অতিরিক্ত কলাম সরিয়ে ডেটাসেট হালকা করতে পারি।

৩. মেশিন লার্নিংয়ে ডাইমেনশনালিটি রিডাকশন

কেন এটি পারফরম্যান্স বাড়ায়?

কলাম বা ফিচারের সংখ্যা খুব বেশি হলে তাকে "Curse of Dimensionality" বলে। এতে ডেটা স্পার্স (Sparse) হয়ে যায় এবং মডেল ওভারফিটিং (Overfitting) করে। ডাইমেনশন কমালে কম্পিউটেশন সময় বাঁচে এবং মডেলটি সহজে বোঝা যায়।

পদ্ধতিগুলোর তুলনা:

পদ্ধতি	কাজ করার ধরন	কখন সবচেয়ে কার্যকর?
Feature Selection	অপ্রয়োজনীয় কলামগুলো সরাসরি বাদ দিয়ে দেয়।	যখন অনেক ফালতু বা কাজের সাথে সম্পর্কহীন কলাম থাকে।
PCA (Principal Component Analysis)	অনেকগুলো কলামকে গাণিতিকভাবে মিলিয়ে নতুন ছোট কিছু কলাম তৈরি করে।	যখন কলামগুলোর মধ্যে অনেক বেশি মিল (Correlation) থাকে।
Sampling	পুরো ডেটাসেট থেকে ছোট একটি রিপ্রেজেন্টেটিভ অংশ বেছে নেয়।	যখন ডেটা পয়েন্ট বা রেকর্ডের সংখ্যা (Rows) অনেক বেশি হয়।

যৌক্তিকতা:

- **Feature Selection** তখন ভালো যখন আপনি ডেটার আসল নাম বা অর্থ পরিবর্তন করতে চান না (যেমন: বয়সের কলাম বয়সই থাকবে)।
- **PCA** হাই-পারফরম্যান্স মডেলিংয়ের (যেমন: ইমেজ প্রসেসিং) জন্য সেরা যেখানে কলামের নাম নয়, বরং তথ্যের গাণিতিক গুরুত্ব বেশি।
- **Sampling** ব্যবহার করা হয় ডেটার আকার (Volume) কমানোর জন্য, যেন ট্রায়াল বা প্রোটোটাইপিং দ্রুত করা যায়।

এই উত্তরগুলো তুমি যদি সুন্দর করে ডায়াগ্রাম দিয়ে লিখতে পারো, তবে খুব ভালো মার্কস পাবে। তোমার আরও কিছু বোঝার থাকলে জিজ্ঞেস করতে পারো!

1. Data Quality Issues & Preprocessing Strategy

Problem Understanding (কি সমস্যা আছে?)

এই dataset-এ তিন ধরনের data quality issue আছে:

1. Missing Values (Incomplete Data)

- Age missing
👉 Example: age = NULL
-

2. Inconsistent Data (Inconsistency)

- Age vs Date of Birth mismatch
👉 Example:
Age = 25, DOB = 1990 ❌
-

3. Noisy Data (Error / Outlier)

- Negative salary
👉 Example: Salary = -5000 ❌
-

Proposed Data Preprocessing Strategy

Step 1: Handling Missing Values

👉 Technique:

- Mean / Class-wise mean

📌 Justification:

- Age numeric → mean use করা যায়
 - যদি class থাকে (e.g., profession), class-wise mean আরও accurate
-

Step 2: Handling Inconsistency

👉 Technique:

- **Data Standardization + Validation Rules**

📌 Example:

- DOB থেকে age calculate করে fix করা

👉 Rule:

- Age = Current Year – Birth Year

📌 Justification:

- logical consistency ensure হয়
-

🔵 Step 3: Handling Noisy Data

👉 Technique:

- **Outlier Detection + Removal**

📌 Example:

- Salary < 0 → remove বা correct

👉 Method:

- Binning / Clustering / Rule-based

📌 Justification:

- ভুল data model accuracy কমায়
-

🧠 Final Conclusion



এই dataset-এ incomplete, noisy, এবং inconsistent data আছে।

Proper preprocessing (mean filling, validation, outlier removal) করলে data quality improve হবে এবং mining result accurate হবে।

2. Data Integration Challenges & Solutions

Challenges in Data Integration

1. Schema Difference

- customer vs cust_id vs client
-

2. Unit Difference (Semantic Heterogeneity)

- Height = 175 cm vs 1.75 m
-

3. Duplicate Records

- একই customer multiple times
-

4. Redundant Attributes

- Monthly income & Annual income
-

Solutions (How to Handle?)

1. Schema Integration

 Technique:

- attribute mapping using metadata

 Example:

- surname = family_name

 Reason:

- same entity identify করা যায়

2. Handling Unit Inconsistency

👉 Technique:

- **Data Transformation (Normalization / Conversion)**

📌 Example:

- 1.75 m → 175 cm

✓ Reason:

- uniform representation নিশ্চিত করে
-

3. Duplicate Detection

👉 Technique:

- Record matching (ID, name)

📌 Example:

- Same ID → duplicate remove

✓ Reason:

- redundancy কমে
-

4. Correlation Analysis

👉 Technique:

- Pearson correlation

📌 Example:

- Monthly income vs Annual income

👉 যদি $r \approx 1 \rightarrow$ highly correlated

✓ Reason:

- একটি attribute remove করা যায়

5. Covariance Analysis

👉 Purpose:

- attributes এর dependency বোঝা

✓ Reason:

- redundant feature detect করা সহজ
-

Final Conclusion



Schema integration, unit conversion, duplicate removal এবং correlation analysis ব্যবহার করে redundancy ও inconsistency কমানো যায়, যা data quality এবং mining performance improve করে।

3. Dimensionality Reduction & Model Performance

Problem Understanding

👉 Dataset-এ 100 attributes আছে

👉 অনেক irrelevant + redundant

👉 Problem:

- High dimensionality → model slow + overfitting
-

Why Dimensionality Reduction Important?

✓ Benefits:

- Processing fast
- Overfitting কমে
- Accuracy improve হয়
- Visualization সহজ হয়

Techniques Comparison

1. Feature Selection

Concept:

- important features select করা
- irrelevant remove করা

Example:

- Age, Salary রাখা
- ID, random noise remove

Best when:

- অনেক irrelevant features আছে

Advantage:

- simple & interpretable
-

2. PCA (Principal Component Analysis)

Concept:

- original features $\rightarrow$ new components
- variance maximize করা

Example:

- 100 feature $\rightarrow$ 10 principal components

Best when:

- features highly correlated

Advantage:

- dimensionality drastically কমায়ে

⚠ Limitation:

- interpretation কঠিন
-

🔵 3. Sampling

🔍 Concept:

- dataset size কমানো

📌 Example:

- 1 million → 100k samples

✓ Best when:

- dataset খুব বড়

✓ Advantage:

- faster computation

⚠ Limitation:

- information loss হতে পারে
-

🔷 Comparison Table

Technique	Use Case	Advantage
Feature Selection	irrelevant features বেশি	easy & interpretable
PCA	correlated features	strong reduction
Sampling	huge dataset	fast processing

🧠 Final Conclusion



Dimensionality reduction model performance improve করে by reducing complexity, removing noise, and improving generalization।

- Feature Selection → best for irrelevant data
- PCA → best for correlated data
- Sampling → best for large datasets

Exam Finishing Line (সব প্রশ্নে ব্যবহার করতে পারো)



“Effective data preprocessing and dimensionality reduction improve data quality, reduce complexity, and significantly enhance the performance and accuracy of data mining models.”

Data Preprocessing — Transformation, Normalization, Discretization & Concept Hierarchy

PART 1: Data Transformation — কী এবং কেন?

Easy Definition:

Data Transformation = পুরনো data-র values গুলোকে নতুন, আরও useful values-এ convert করা — এমনভাবে যেন প্রতিটা পুরনো value-এর জন্য একটা নতুন value থাকে।

 সহজ ভাষায়:

ধরো তোমার কাছে একটা dataset আছে যেখানে মানুষের income আছে — কেউ ১২,০০০ টাকা পায়, কেউ ৯৮,০০০ টাকা পায়। এই বড় বড় সংখ্যাগুলো নিয়ে directly কাজ করা কঠিন। তাই আমরা এগুলোকে **transform** করে ছোট, manageable range-এ নিয়ে আসি।

Data Transformation-এর Methods (৫টা):

Method	কী করে
Smoothing	Data থেকে noise (ভুল/অস্বাভাবিক values) সরিয়ে দেয়
Attribute Construction	পুরনো attributes থেকে নতুন attribute তৈরি করে
Aggregation	Data summarize করে (যেমন: data cube)
Normalization	Data-কে একটা ছোট range-এ scale করে
Discretization	Continuous data-কে intervals/categories-এ ভাগ করে

NORMALIZATION:

তাহলে আসলে ৪টা **unique method** আছে:

1. Min-Max Normalization
2. Z-Score / Standard Scaler (same thing)
3. Robust Scaler
4. Decimal Scaling

চলো একটা একটা করে কখন ব্যবহার হয় + **math** সহ বুঝি।

একটা **Common Dataset** নেই সবগুলোতে — তাহলে **compare** করা সহজ হবে:

Dataset (মানুষের বয়স):

Values: 20, 25, 30, 35, 200 ← 200 হলো outlier (অস্বাভাবিক বড়)

এই একই data-তে সব ৪টা method apply করবো।

◆ Method 1: Min-Max Normalization

✓ **Easy Definition:**

Data-কে টেনে একটা নির্দিষ্ট **range**-এ (সাধারণত 0 থেকে 1) নিয়ে আসা।

📐 **Formula:**

$$v' = (v - \min) / (\max - \min)$$

নতুন range [0,1] হলে এটুকুই। [a,b] হলে শেষে $\times (b-a) + a$ যোগ হয়।

🧮 **Math:**

min = 20, max = 200

$$20 \rightarrow (20 - 20) / (200 - 20) = 0/180 = 0.000$$

$$25 \rightarrow (25 - 20) / (200 - 20) = 5/180 = 0.028$$

$$30 \rightarrow (30 - 20) / (200 - 20) = 10/180 = 0.056$$

$$35 \rightarrow (35 - 20) / (200 - 20) = 15/180 = 0.083$$

$$200 \rightarrow (200 - 20) / (200 - 20) = 180/180 = 1.000$$

😬 সমস্যা দেখো:

20, 25, 30, 35 সবগুলো **0.000** থেকে **0.083** এর মধ্যে চুপসে গেছে! কারণ outlier 200 পুরো scale নিয়ে নিয়েছে। Normal values গুলোর মধ্যে পার্থক্য বোঝা যাচ্ছে না।

 কখন ব্যবহার করবে?

পরিস্থিতি

ব্যবহার করবে?

Data-র exact range জানা আছে

✓ হ্যাঁ

কোনো outlier নেই

✓ হ্যাঁ

Neural Network / Image processing

✓ হ্যাঁ (pixel 0-255 → 0-1)

Outlier আছে dataset-এ

✗ না

◆ Method 2: Z-Score Normalization (= Standard Scaler)

✓ Easy Definition:

প্রতিটা value থেকে **mean** বাদ দিয়ে **standard deviation** দিয়ে ভাগ করা — বলে দেয় value টা গড় থেকে কত "দূরে"।

 Formula:

$$v' = (v - \mu) / \sigma$$

μ = mean (গড়)

σ = standard deviation

 Math:

Values: 20, 25, 30, 35, 200

Step 1 — Mean বের করো:

$$\mu = (20 + 25 + 30 + 35 + 200) / 5$$

$$\mu = 310 / 5$$

$$\mu = 62$$

Step 2 — Standard Deviation বের করো:

প্রতিটার জন্য $(v - \mu)^2$:

$$(20-62)^2 = (-42)^2 = 1764$$

$$(25-62)^2 = (-37)^2 = 1369$$

$$(30-62)^2 = (-32)^2 = 1024$$

$$(35-62)^2 = (-27)^2 = 729$$

$$(200-62)^2 = (138)^2 = 19044$$

$$\text{Variance} = (1764+1369+1024+729+19044) / 5$$

$$= 23930 / 5$$

$$= 4786$$

$$\sigma = \sqrt{4786} \approx 69.18$$

Step 3 — Z-score বের করো:

$$\begin{aligned} 20 &\rightarrow (20 - 62) / 69.18 = -42/69.18 = -0.607 \\ 25 &\rightarrow (25 - 62) / 69.18 = -37/69.18 = -0.535 \\ 30 &\rightarrow (30 - 62) / 69.18 = -32/69.18 = -0.463 \\ 35 &\rightarrow (35 - 62) / 69.18 = -27/69.18 = -0.390 \\ 200 &\rightarrow (200 - 62) / 69.18 = 138/69.18 = +1.994 \end{aligned}$$

🤖 সমস্যা দেখো:

Outlier 200 আছে বলে **mean 62** হয়ে গেছে — যেটা আসল "সাধারণ" values (20-35) থেকে অনেক দূরে। Mean নিজেই outlier দ্বারা distorted।

🕒 কখন ব্যবহার করবে?

পরিস্থিতি	ব্যবহার করবে?
Outlier কম বা নেই	✅ হ্যাঁ
SVM, Logistic Regression, PCA	✅ হ্যাঁ
Data normally distributed হলে	✅ হ্যাঁ
Heavy outlier আছে	⚠️ সতর্কতার সাথে

📦 Method 3: Robust Scaler

✅ Easy Definition:

Median এবং **IQR (Interquartile Range)** ব্যবহার করে scale করা — outlier এর কোনো effect নেই কারণ median/IQR outlier দেখে না।

📐 Formula:

$$v' = (v - \text{median}) / \text{IQR}$$

$$\text{IQR} = Q3 - Q1$$

Q1 = 25th percentile (নিচের দিক থেকে ২৫%)

Q3 = 75th percentile (নিচের দিক থেকে ৭৫%)

📊 Math:

Values sorted: 20, 25, 30, 35, 200

Step 1 — Median বের করো:

5টা value → মাঝেরটা = 30

Median = 30

Step 2 — Q1 এবং Q3 বের করো:
Lower half (median বাদে): 20, 25
 $Q1 = (20+25)/2 = 22.5$

Upper half (median বাদে): 35, 200
 $Q3 = (35+200)/2 = 117.5$

$IQR = Q3 - Q1 = 117.5 - 22.5 = 95$

Step 3 — Scale করো:
 $20 \rightarrow (20 - 30) / 95 = -10/95 = -0.105$
 $25 \rightarrow (25 - 30) / 95 = -5/95 = -0.053$
 $30 \rightarrow (30 - 30) / 95 = 0/95 = 0.000$
 $35 \rightarrow (35 - 30) / 95 = 5/95 = +0.053$
 $200 \rightarrow (200 - 30) / 95 = 170/95 = +1.789$

✅ দেখো পার্থক্য:

Normal values (20-35) এখন **-0.105** থেকে **+0.053** — অনেক কাছাকাছি এবং meaningful! Outlier 200 আলাদা থাকলো কিন্তু normal data নষ্ট হলো না।

🕒 কখন ব্যবহার করবে?

পরিস্থিতি	ব্যবহার করবে?
Outlier আছে dataset -এ	✅ সবচেয়ে ভালো choice
Medical data, Financial data	✅ হ্যাঁ (outlier common)
Outlier কে ignore করতে চাও	✅ হ্যাঁ
Data normally distributed	⚠️ Z-score বেশি ভালো

◆ Method 4: Decimal Scaling

✅ Easy Definition:

Value-কে **10** এর উপযুক্ত **power** দিয়ে ভাগ করে -1 থেকে 1 এর মধ্যে নিয়ে আসা — শুধু decimal point সরানো হয়।

📐 Formula:

$$v' = v / 10^j$$

j = সবচেয়ে ছোট integer যাতে $\text{Max}(|v'|) < 1$

🧮 Math:

Values: 20, 25, 30, 35, 200

সবচেয়ে বড় absolute value = 200

$j=1 \rightarrow 200/10 = 20 \rightarrow \text{এখনো } \geq 1$ ❌

$j=2 \rightarrow 200/100 = 2.0 \rightarrow \text{এখনো } \geq 1$ ❌

$j=3 \rightarrow 200/1000 = 0.2 \rightarrow 1 \text{ এর কম}$ ✅

তাহলে $j = 3$, সবাইকে 1000 দিয়ে ভাগ:

$20 \rightarrow 20/1000 = 0.020$

$25 \rightarrow 25/1000 = 0.025$

$30 \rightarrow 30/1000 = 0.030$

$35 \rightarrow 35/1000 = 0.035$

$200 \rightarrow 200/1000 = 0.200$

🕒 কখন ব্যবহার করবে?

পরিস্থিতি	ব্যবহার করবে?
Simple, fast normalization দরকার	✅ হ্যাঁ
Data-র magnitude বড় (হাজার, লক্ষ)	✅ হ্যাঁ
Quick preprocessing	✅ হ্যাঁ
Precise scaling দরকার	❌ না

🎯 সব একসাথে **Compare** করো (Same Data):

Value	Min-Max	Z-Score	Robus t	Decimal
20	0.000	-0.607	-0.105	0.020
25	0.028	-0.535	-0.053	0.025
30	0.056	-0.463	0.000	0.030
35	0.083	-0.390	+0.053	0.035
200	1.000	+1.994	+1.789	0.200

🧠 কখন কোনটা **USE** করবে — চিট শিট:

Outlier আছে?

— হ্যাঁ $\rightarrow$ ROBUST SCALER ✅ (best choice)

— না $\rightarrow$ Data কেমন?

— Normal distribution $\rightarrow$ Z-SCORE ✅

— Fixed range দরকার (0-1) $\rightarrow$ MIN-MAX ✅

— শুধু quick/simple করতে চাও → DECIMAL SCALING ✓

Method	Best For	Avoid When
Min-Max	Neural Network, Image data, fixed range দরকার	Outlier থাকলে
Z-Score	SVM, PCA, Logistic Regression	Heavy outlier থাকলে
Robust	Medical, Financial data যেখানে outlier common	Data perfectly normal হলে
Decimal	Quick preprocessing, large magnitude data	Precise scaling দরকার হলে

◆ কখন Normalization করবো, কখন Standardization?

এটাই সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন। চলো **situation** দেখে বুঝি —

✓ NORMALIZATION করবো যখন —

1 Algorithm-এর input একটা fixed range-এ থাকতে হবে

Neural Network, Deep Learning — এগুলো 0 to 1 range-এ ভালো কাজ করে।

উদাহরণ:

Image pixel values: 0 থেকে 255

Min-Max করলে → 0 থেকে 1

Neural network এখন সহজে process করতে পারবে ✓

2 Data-র distribution জানো না

Data কোনো specific pattern follow করছে না — just range fix করতে চাও।

3 Distance-based algorithm ব্যবহার করছে

KNN, K-Means — এগুলো দুটো point এর মধ্যে **distance** মাপে। বড় range মানে বেশি influence।

উদাহরণ:

Feature 1 — Age: 20 থেকে 60

Feature 2 — Income: 10,000 থেকে 1,00,000

Normalize না করলে KNN শুধু Income দেখে distance মাপবে

Age-কে practically ignore করবে ✗

Normalize করলে দুটো সমান weight পাবে ✓

4 Outlier নেই dataset-এ

Data clean, কোনো অস্বাভাবিক value নেই।

✓ STANDARDIZATION করবো যখন —

1 Algorithm assume করে data normally distributed

অনেক ML algorithm ধরে নেয় data bell-curve shape-এ আছে।

এই algorithms-এ Standardization ব্যবহার করো:

- SVM (Support Vector Machine)
- Logistic Regression
- Linear Regression
- PCA (Principal Component Analysis)

2 Data-তে Outlier আছে

Z-Score এ outlier থাকলেও বাকি data নষ্ট হয় না (Min-Max এর মতো সব চূপসে যায় না)।

উদাহরণ:

Salary: 20K, 25K, 30K, 35K, 500K ← outlier

Min-Max করলে: 20K-35K সব 0.00-0.03 এ চূপসে যাবে ✗

Z-Score করলে: 20K-35K এর মধ্যে পার্থক্য বোঝা যাবে ✓

3 Features-এর unit আলাদা আলাদা

উদাহরণ:

Height: meter এ → 1.5, 1.7, 1.8

Weight: kg এ → 50, 70, 90

Age: years এ → 20, 35, 60

এদের directly compare করা যায় না

Standardization করলে সবার unit চলে যায়,

শুধু "গড় থেকে কতটা দূরে" সেটা থাকে ✓

4 Data-র exact range জানো না বা range bounded না

কোনো theoretical minimum বা maximum নেই।

🎯 Decision Chart — কোনটা ব্যবহার করবো?

Algorithm কী ব্যবহার করছো?

- Neural Network / Deep Learning / Image
 - NORMALIZATION (Min-Max, 0-1) ✓
- KNN / K-Means (distance-based)
 - NORMALIZATION (Min-Max) ✓

- SVM / Logistic Regression / PCA / Linear Regression
→ STANDARDIZATION (Z-Score) ✓
- Outlier আছে heavily?
 - হ্যাঁ + range fix চাই → ROBUST SCALER ✓
 - হ্যাঁ + distribution চাই → Z-SCORE ✓
- শুধু quick scale করতে চাই, magnitude বড়
→ DECIMAL SCALING ✓

PART 3: Discretization — বিস্তারিত

✓ Easy Definition:

Discretization = Continuous (অবিচ্ছিন্ন) data-কে ছোট ছোট **intervals** বা **buckets**-এ ভাগ করা।

🧠 সহজ উদাহরণ:

Age একটা continuous value — কেউ 18, কেউ 25, কেউ 67 বছর। Discretization করলে:

- 0-18 → "Child"
- 19-35 → "Young Adult"
- 36-60 → "Adult"
- 60+ → "Senior"

এখন সংখ্যার বদলে **label** ব্যবহার করা যাচ্ছে।

📌 কেন দরকার?

- Data size কমেয়
- Classification algorithm-এর জন্য সহজ হয়
- Pattern বোঝা সহজ হয়

📌 দুটো ভাগ:

	Supervised	Unsupervised
Class label লাগে?	হ্যাঁ	না
উদাহরণ	Decision Tree, Chi-merge	Binning, Histogram, Clustering

Split vs Merge:

- **Top-down Split** = বড় range কে ভেঙে ছোট করে
 - **Bottom-up Merge** = ছোট intervals জোড়া লাগিয়ে বড় করে
-

♦ Discretization Methods — ৫টা:

1 Binning (Unsupervised, Top-down Split)

✓ Easy Definition:

Data-কে সমান ভাগে (bins/buckets) ভাগ করা এবং প্রতি bin-এর values গুলোকে smooth করা।

Example:

Data: 4, 8, 15, 21, 21, 24, 25, 28, 34

3টা bin-এ ভাগ করো:

- Bin 1: 4, 8, 15
- Bin 2: 21, 21, 24
- Bin 3: 25, 28, 34

Smoothing by bin mean:

- Bin 1 mean = $(4+8+15)/3 = 9 \rightarrow$ সব values হয় 9
- Bin 2 mean = $(21+21+24)/3 = 22 \rightarrow$ সব values হয় 22
- Bin 3 mean = $(25+28+34)/3 = 29 \rightarrow$ সব values হয় 29

2 Histogram Analysis (Unsupervised, Top-down Split)

✓ Easy Definition:

Data-এর frequency distribution দেখে **automatically intervals** তৈরি করা।

 ভাবো:

Histogram এ দেখা যাচ্ছে 0-20 range এ বেশি data, 20-40 এ কম — সেই অনুযায়ী intervals ঠিক হয়।

3 Clustering Analysis (Unsupervised, Split বা Merge)

✓ Easy Definition:

Similar values গুলোকে automatically একই group/cluster-এ রাখা।

 ভাবো:

Income data: [12K, 15K, 14K, 80K, 85K, 90K] Clustering বলবে: প্রথম তিনটা একটা cluster (low income), পরের তিনটা আরেকটা cluster (high income)।

4 Decision Tree Analysis (Supervised, Top-down Split)

✓ Easy Definition:

Class label জেনে data-কে এমনভাবে split করা যাতে প্রতিটা interval এ mostly একটাই class থাকে।

🧠 উদাহরণ:

Cancer detection: age দিয়ে "cancerous" vs "benign" ভাগ করতে decision tree বলবে — "age > 50 হলে cancerous এর chance বেশি" → সেখানেই split।

5 Chi-merge / Correlation Analysis (Supervised, Bottom-up Merge)

✓ Easy Definition:

χ^2 (Chi-square) test ব্যবহার করে দেখা যে দুটো neighboring intervals এর class distribution কতটা similar — similar হলে merge করো।

🧠 Logic:

- যদি দুটো interval এ same রকম class distribution থাকে → এরা আলাদা থাকার দরকার নেই → **merge** করো
- যদি আলাদা distribution থাকে → আলাদাই থাকবে
- **Low χ^2 value** = similar distribution = merge করো

🔄 Process:

শুরুতে প্রতিটা value আলাদা interval → ধীরে ধীরে similar গুলো merge হতে থাকে → যতক্ষণ না stopping condition পূরণ হয়।

◆ PART 4: Concept Hierarchy Generation

✓ Easy Definition:

Concept Hierarchy = Data-এর values গুলোকে ছোট থেকে বড় **concept**-এ সাজানো — যেমন specific থেকে general।

🧠 সহজ উদাহরণ:

Street → City → State → Country

(সবচেয়ে specific) (সবচেয়ে general)

"Dhanmondi" → "Dhaka" → "Dhaka Division" → "Bangladesh"

📌 কেন দরকার?

Data Warehouse-এ **drill down** (বিস্তারিত দেখা) এবং **roll up** (summarize দেখা) করতে ব্যবহার হয়।

- Roll up: City level থেকে Country level এ যাওয়া
- Drill down: Country থেকে City তে নামা

◆ Concept Hierarchy তৈরির ৪টা উপায়:

1 Schema Level Ordering (User/Expert দেয়)

Schema-তেই বলা থাকে কোনটা কোনটার উপরে:

street < city < state < country

2 Explicit Data Grouping

নির্দিষ্ট values গুলোকে manually group করা:

{Urbana, Champaign, Chicago} → Illinois

মানে এই তিনটা city মিলে Illinois state।

3 Partial Ordering

শুধু কিছু level specify করা:

street < city (বাকিটা specify করা হয়নি)

4 Automatic Generation (সবচেয়ে গুরুত্বপূর্ণ!)

✓ Easy Definition:

প্রতিটা attribute-এ কতটা **distinct values** আছে তা গুনে automatically hierarchy তৈরি করা।

🧠 Rule:

যে **attribute**-এ সবচেয়ে বেশি **distinct value** → সেটা **hierarchy**-র সবচেয়ে নিচে

📊 Example (Slide থেকে):

Attribute	Distinct Values	Hierarchy Level
Country	15	সবচেয়ে উপরে (most general)
Province/State	365	উপর থেকে ২য়

City	3,567	নিচ থেকে ২য়
Street	6,74,339	সবচেয়ে নিচে (most specific)

Logic: Street-এ লক্ষ লক্ষ আলাদা নাম → এটা সবচেয়ে specific | Country-তে মাত্র 15টা → এটা সবচেয়ে general |

Quick Revision Summary

DATA TRANSFORMATION

- |
- | — Normalization (range-এ আনো)
 - | | — Min-Max → নির্দিষ্ট range [a, b]
 - | | — Z-Score → mean/std দিয়ে
 - | | — Decimal Scaling → 10^j দিয়ে ভাগ
- |
- | — Discretization (intervals-এ ভাগ করো)
 - | | — Unsupervised: Binning, Histogram, Clustering
 - | | — Supervised: Decision Tree, Chi-merge
- |
- | — Concept Hierarchy (specific → general)
 - | — Manual: Schema, Grouping, Partial
 - | — Automatic: Distinct value count দেখে